

CS310 Natural Language Processing

自然语言处理

Lecture 05 - Transformer

Instructor: Yang Xu

主讲人：徐炆

xuyang@sustech.edu.cn

Overview

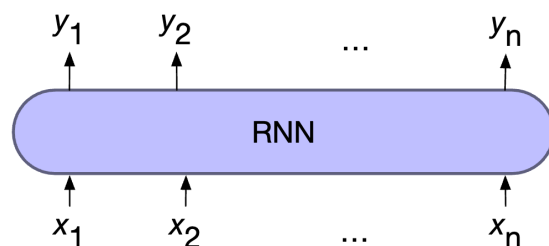
- Motivation
- Transformer Basics
- Transformer Details
- Extensions

Overview

- **Motivation**
 - **RNNs, Encoder-Decoder, and Attention**
- Transformer Basics
- Transformer Details
- Extensions

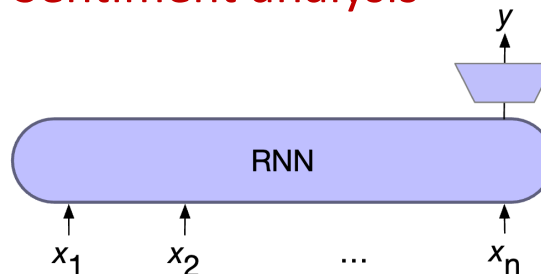
Common RNN Architecture for NLP

POS tagging, NER

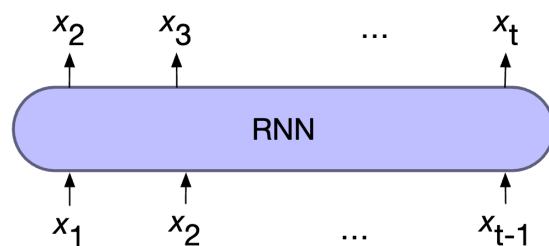


a) sequence labeling

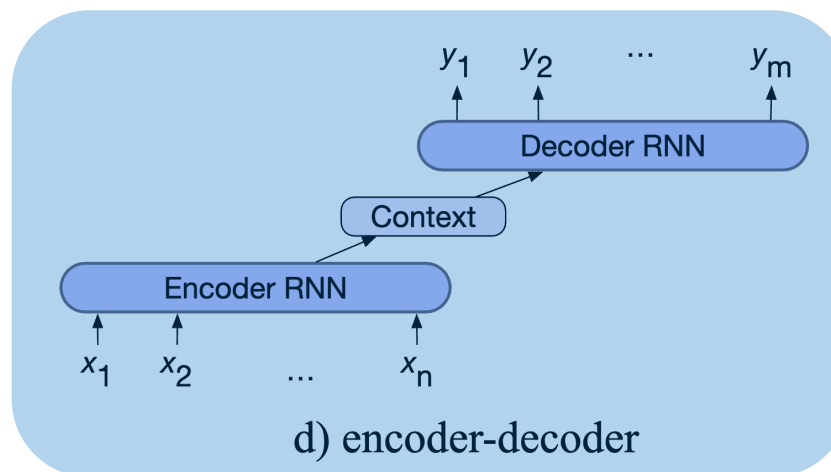
Sentiment analysis



b) sequence classification



c) language modeling



d) encoder-decoder

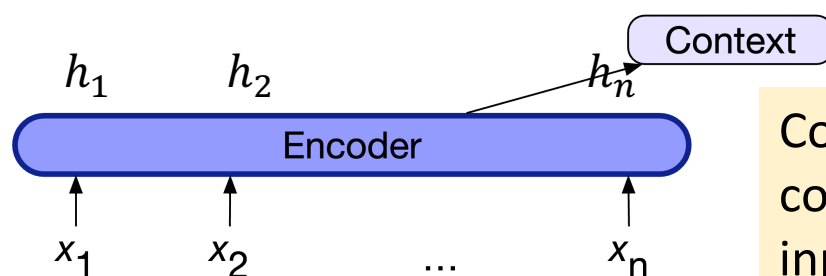
Encoder-decoder: also called **sequence-to-sequence** network

- Take an input sequence and translate it to an output sequence of *different* length
- Applications: summarization, question answering, dialogue, and particularly useful for *machine translation*

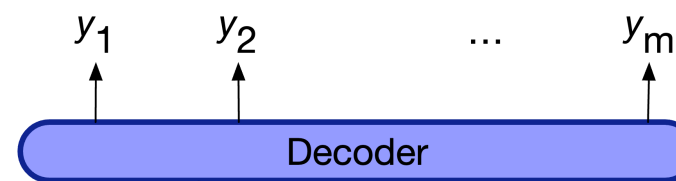
Encoder-Decoder

- Three conceptual components: an **encoder**, a **context** vector, and a **decoder**

Encoder accepts an input sequence $x_{1:n}$ and generates a sequence of contextualized representations $h_{1:n}$



Context vector $c = f(h_{1:n})$ conveys the information of input to the decoder



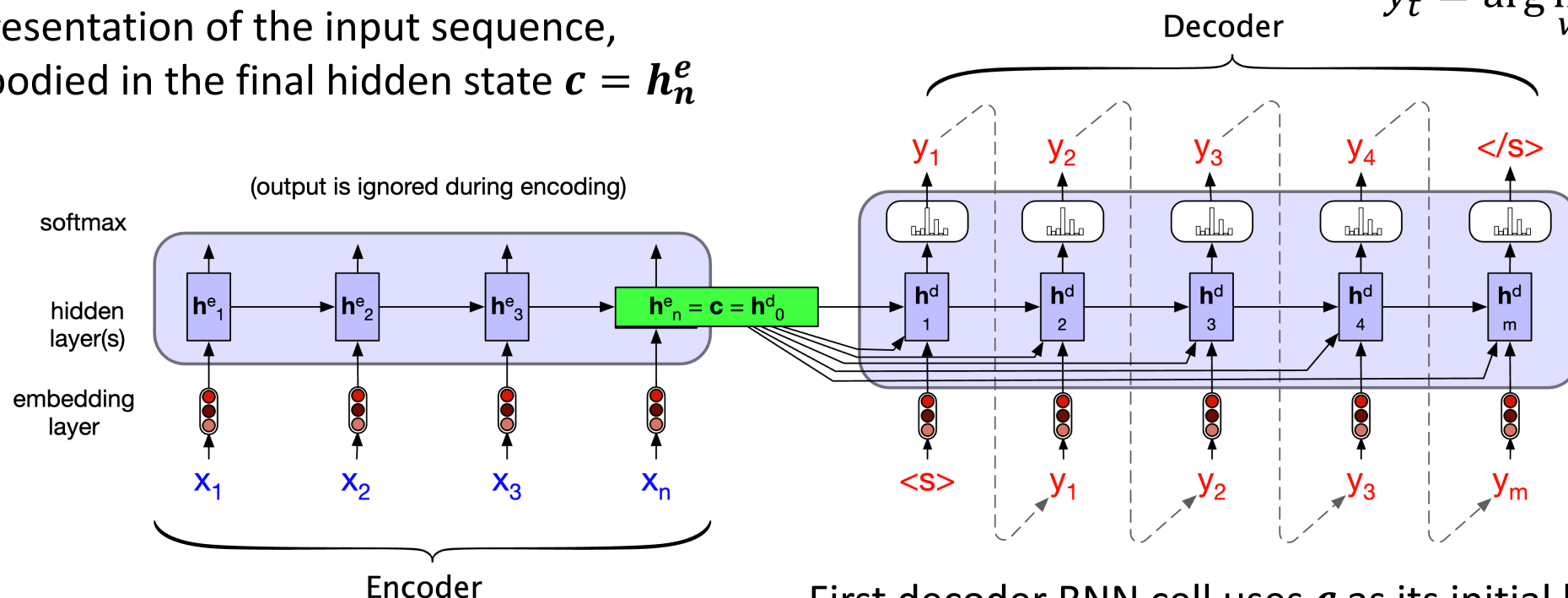
Decoder accepts c as input and generate a sequence of new representations $h'_{1:m}$, from which an output sequence $y_{1:m}$ is obtained

Encoder-Decoder RNN for Machine Translation

Encoder generates a contextualized representation of the input sequence, embodied in the final hidden state $\mathbf{c} = \mathbf{h}_n^e$

$$\mathbf{h}_t^d = g(y_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \quad \hat{y}_t = \text{softmax}(f(\mathbf{h}_t^d))$$

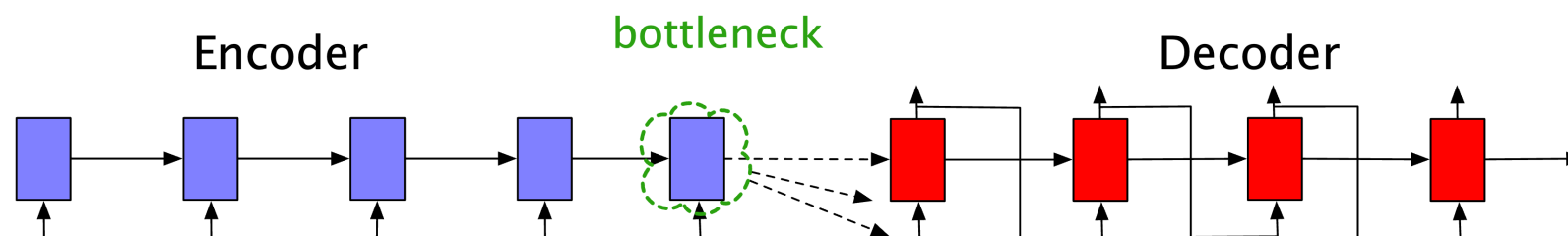
$$y_t = \arg \max_{w \in V} \hat{y}_t$$



First decoder RNN cell uses $\mathbf{c}$ as its initial hidden state h_0^d
 $\mathbf{c}$ is made available to all time steps to preserve its influence

Attention in Encoder-Decoder

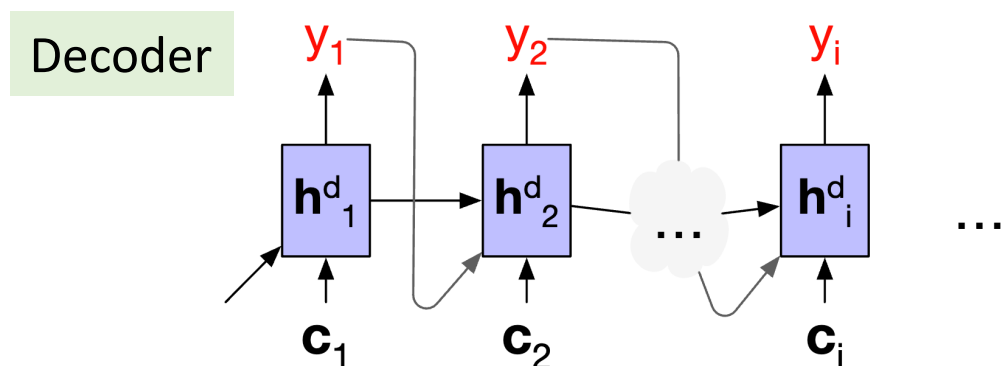
- **Bottleneck problem:** the last hidden state h_n^e from the encoder must represent *everything* about the input meaning -- It is difficult because earlier information at the beginning (especially for long sentence) may not be well represented



- **Attention mechanism** is a solution: allowing the decoder to get information from *all* the hidden states of encoder, not just the last one
- **Idea:** Create the context vector c by taking a *weighted sum* of all the encoder states $h_1^e, \dots, h_n^e$
- The weights attend to a particular part of input that is relevant for the token currently being produced by the decoder

Attention in Encoder-Decoder

- **Attention idea:** replaces the *static* context c with a *dynamically* derived one c_i , which is different for each decoding step i



The context used for decoding at step 1 (c_1) is different from the one for step 2 (c_2)

Hidden state used for decoding at step i :

$$h^d_i = g(y_{i-1}, h^d_{i-1}, c_i)$$

c_i should capture how *relevant* each encoder h^e_j is to the decoder state h^d_{i-1} , using some **score**: $\text{score}(h^d_{i-1}, h^e_j)$

Simplest option -- **dot-product attention**:

$$\text{score}(h^d_{i-1}, h^e_j) = h^d_{i-1} \cdot h^e_j$$

Measures relevance by similarity between vectors

Dot-Product Attention

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$

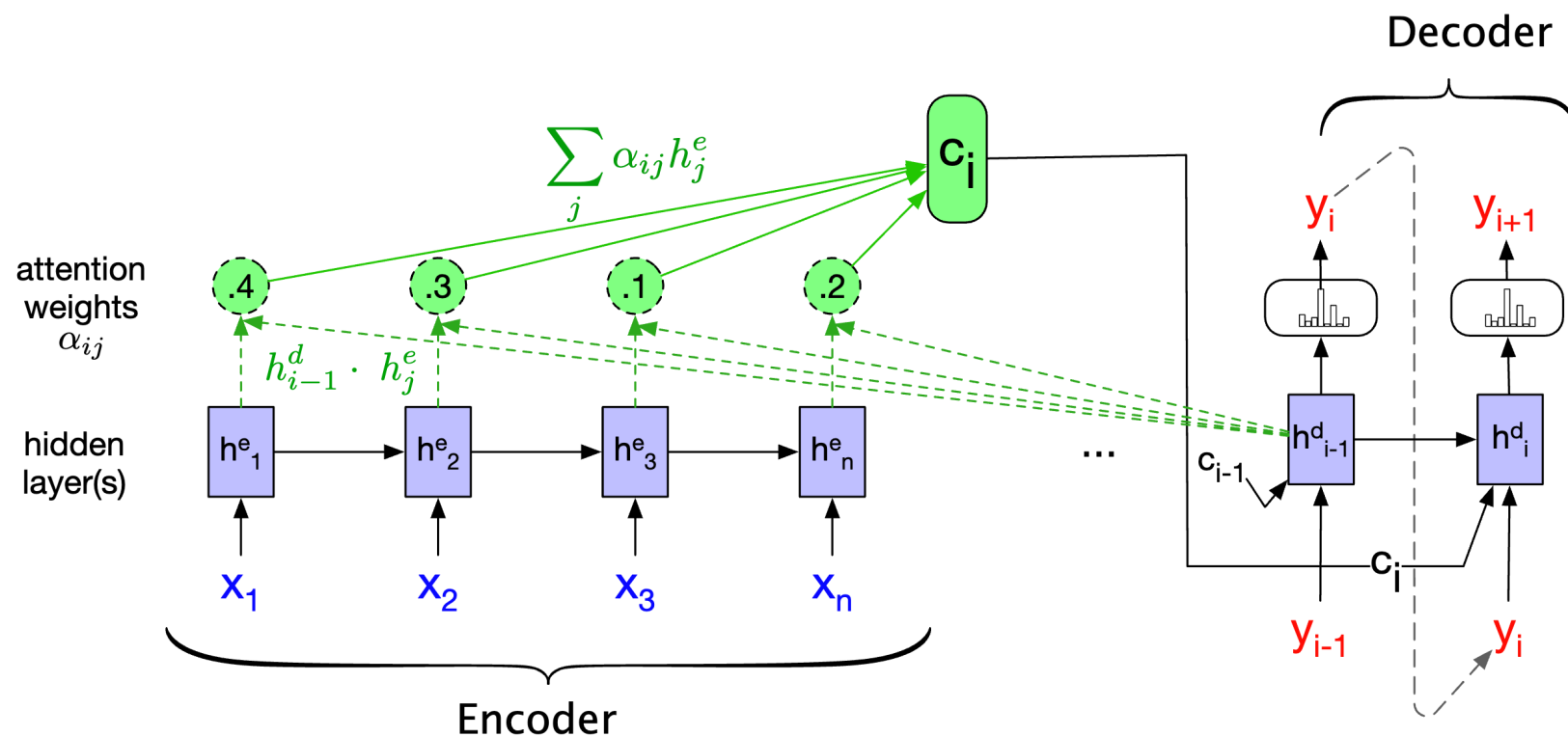
- Score from dot-product reflects the degree of similarity between two vectors
- Normalize the scores with a softmax to obtain vector of weights, α_{ij} , that tells the proportional relevance of each encoder state $\mathbf{h}_j^e$ to the decoder state $\mathbf{h}_{i-1}^d$:

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)) = \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}$$

- Finally, given the α_{ij} , we can compute the dynamic context vector $\mathbf{c}_i$ by taking a weighted average of all encoder hidden states:

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

Encoder-Decoder with Attention



What if h_{i-1}^d, h_j^e are of different dimensions?

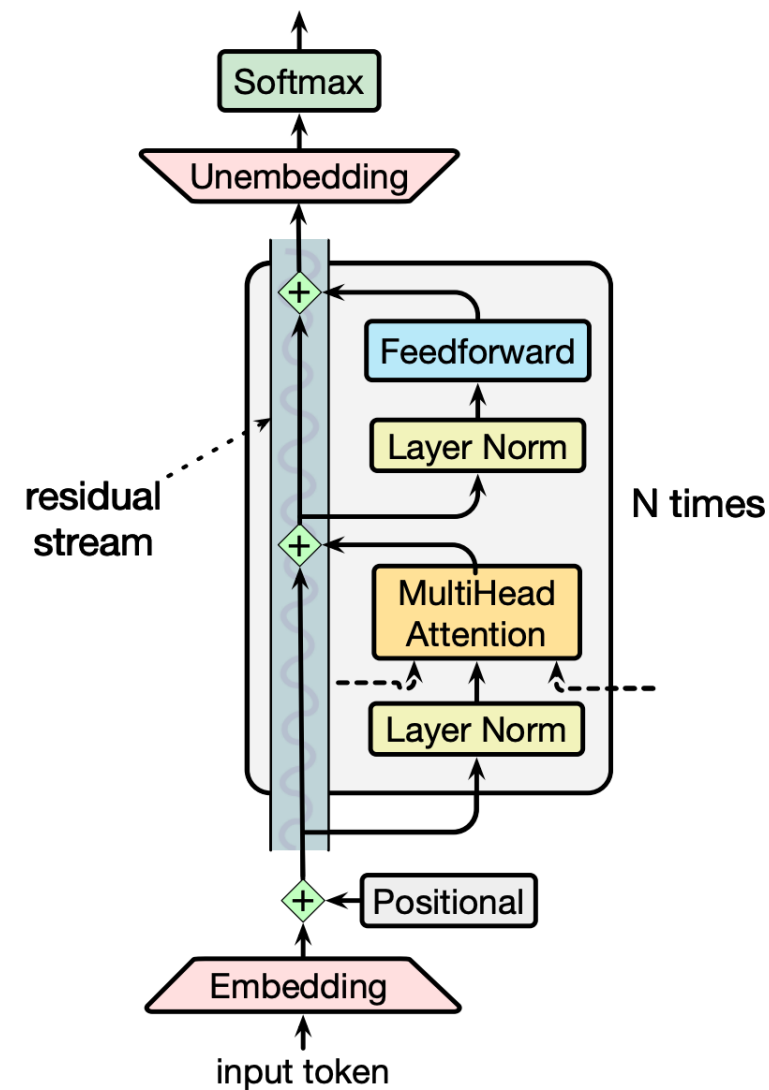
Need a more sophisticated scoring function, such as bilinear:

$$\text{score}(h_{i-1}^d, h_j^e) = h_{i-1}^d W_s h_j^e$$

The **self-attention** defined in transformer architecture is a modification on dot-product attention

Overview

- Motivation
- **Transformer Basics**
 - Self-Attention; Multihead Attention
 - Transformer Blocks
 - Positional Embedding
- Transformer Details
- Extensions

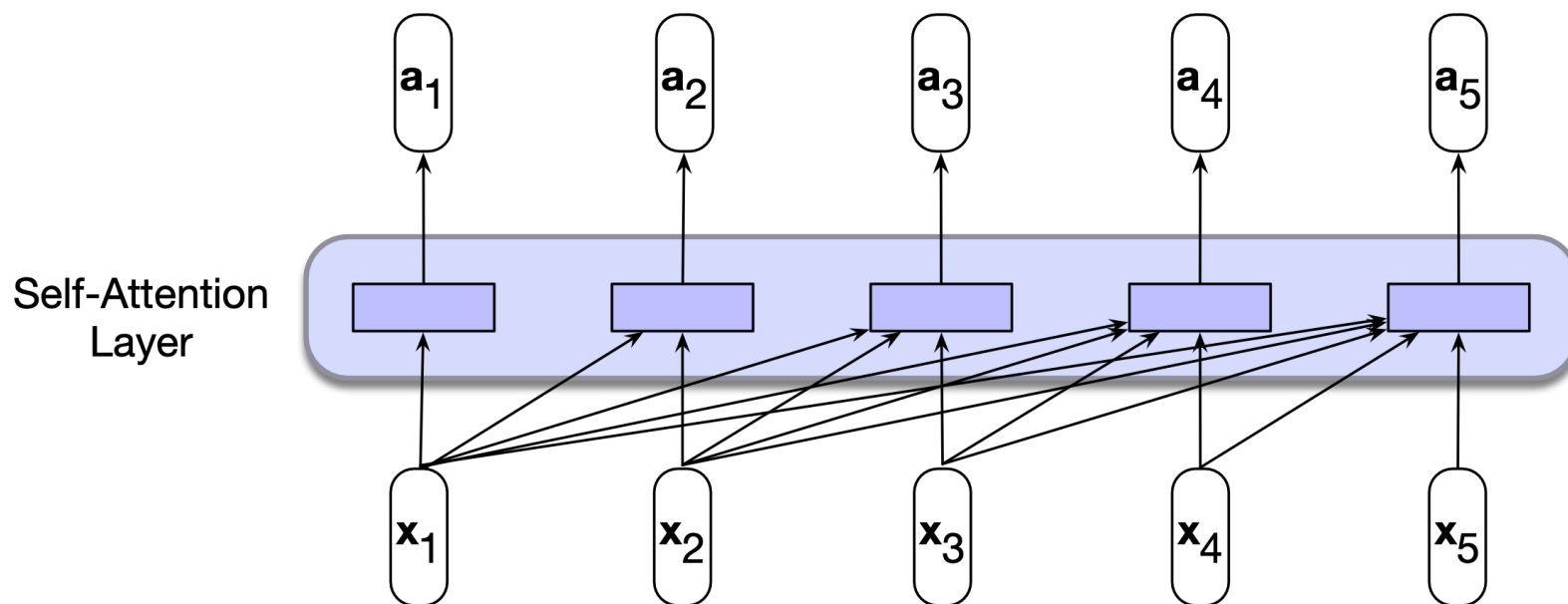


Self-Attention: Motivation

- S1: *The book cover is **orange**.*
- S2: *The book is about **orange**.*
- Which orange means color and which means fruit?
- Needs a mechanism to look at the context to “compute” meanings of words
- **Self-attention** is such a mechanism: given the representations (vectors) of words from previous layer, look broadly in the context and integrate them into the output representations (vectors) for the next layer

Causal Self-Attention: Attending to preceding context

- In **causal**, or **backward looking** self-attention, context is any of the prior words
(In **bidirectional** self-attention, context can include future words)



When processing x_i ,
it has access to $x_1, x_2, \dots, x_i$,
but no access to beyond:
 $x_{i+1}, \dots$

Self-Attention (version 1): Using Dot-Product

- Recall: the intuition of attention is to compare an item with other items in a way to reveal their relevance.
- In causal self-attention, a word is compared to its preceding words; the result is then used to compute the output for current word
- Our old friend **dot-product**:

① $\text{score}(x_i, x_j) = x_i \cdot x_j \quad \forall j \leq i$

② Normalize to a probability distribution:

$$\alpha_{ij} = \text{softmax}(x_i \cdot x_j) = \frac{\exp(x_i \cdot x_j)}{\sum_k \exp(x_i \cdot x_k)} \quad \forall j \leq i$$

③ Compute output a_i as the weighted average: $a_i = \sum_{j \leq i} \alpha_{ij} x_j$

Note: α_{ij} is likely to be highest for $i = j$, i.e., a word is most similar to itself

Self-Attention (version 2): query, key, and value

- Considering **three different roles** each input token plays during the attention process:
- **Query**: Current focus of attention when comparing to all the other preceding inputs;
- **Key**: As a preceding input being compared to;
- **Value**: Contribute “meaning” to the output for the current focus of attention.
- Introduce weight matrices W^Q , W^K , and W^V to project each input vector x_i into a representation of its role as a key, query, and value:

$$q_i = x_i W^Q; \quad k_i = x_i W^K; \quad v_i = x_i W^V$$

Dimensions: $\left. \begin{array}{l} \text{input } x_i \\ \text{output } y_i \\ \text{intermediate } a_i \end{array} \right\} \in \mathbb{R}^d$

$\left. \begin{array}{l} q_i \\ k_i \end{array} \right\} \in \mathbb{R}^{d_k}$

$v_i \in \mathbb{R}^{d_v}$

$W^Q \in \mathbb{R}^{d \times d_k}$

$W^K \in \mathbb{R}^{d \times d_k}$

$W^V \in \mathbb{R}^{d \times d_v}$

In the original work
(Vaswani et al. 2017):
 $d = 512$,
 $d_k = d_v = 64$

Self-Attention (version 2): query, key, and value

- To compute score between a current focus of attention x_i , and a preceding element x_j , use the dot-product between its **query** vector q_i and **key** vector k_j :

$$\text{score}(x_i, x_j) = q_i \cdot k_j$$

- Normalize the scores into α_{ij} using softmax (same as version 1)
- Compute the output a_i as the weighted sum of **value** vectors v :

Note: similar to version 1, α_{ij} is also likely to be highest for $i = j$

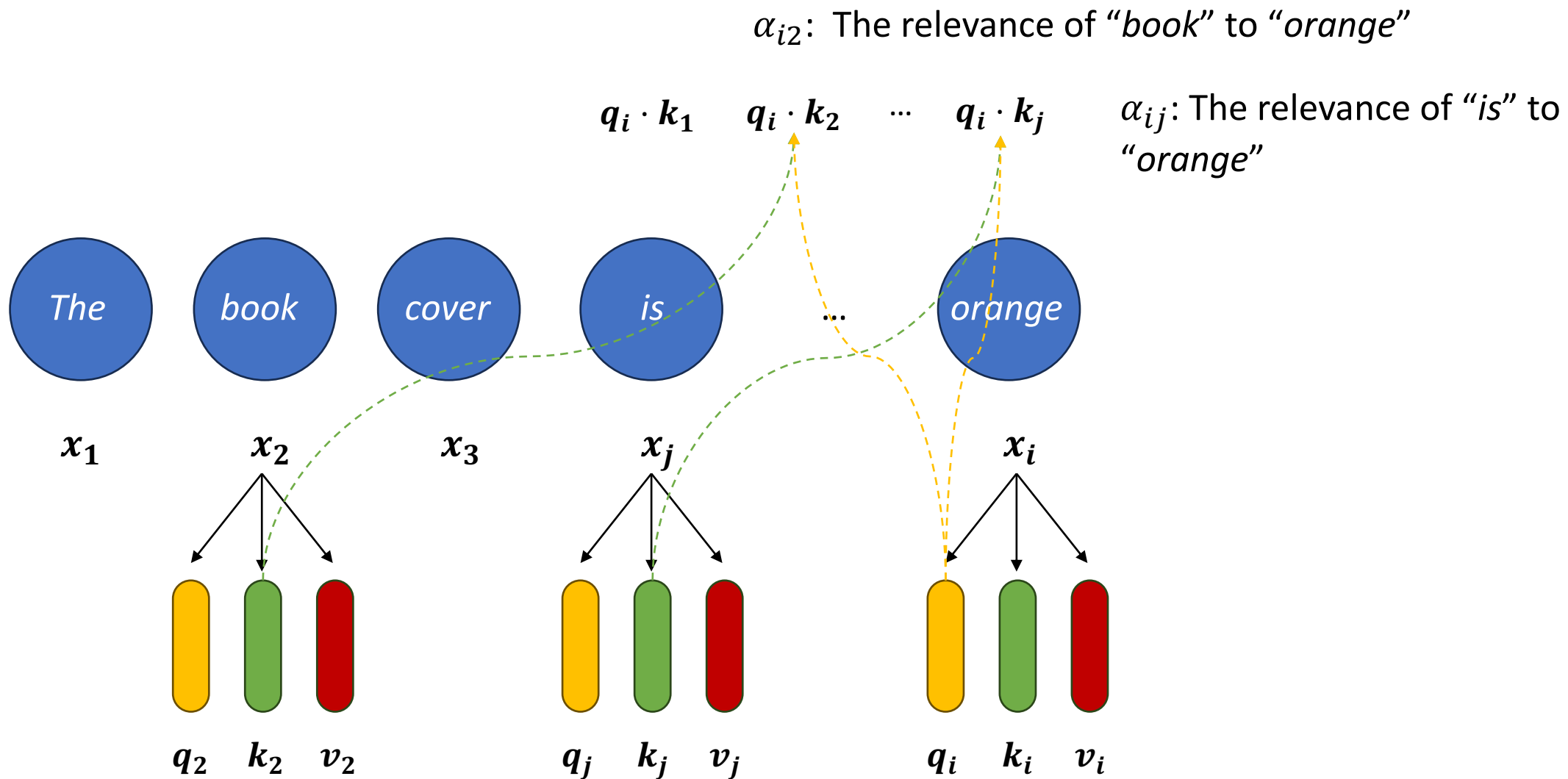
Note: $d = 512$, $d_k = d_v = 64$
How do 64-d vectors (v_j) end up in 512-d (a_i)? -- Concatenation (explained in multihead attention)

$$a_i = \sum_{j \leq i} \alpha_{ij} v_j$$

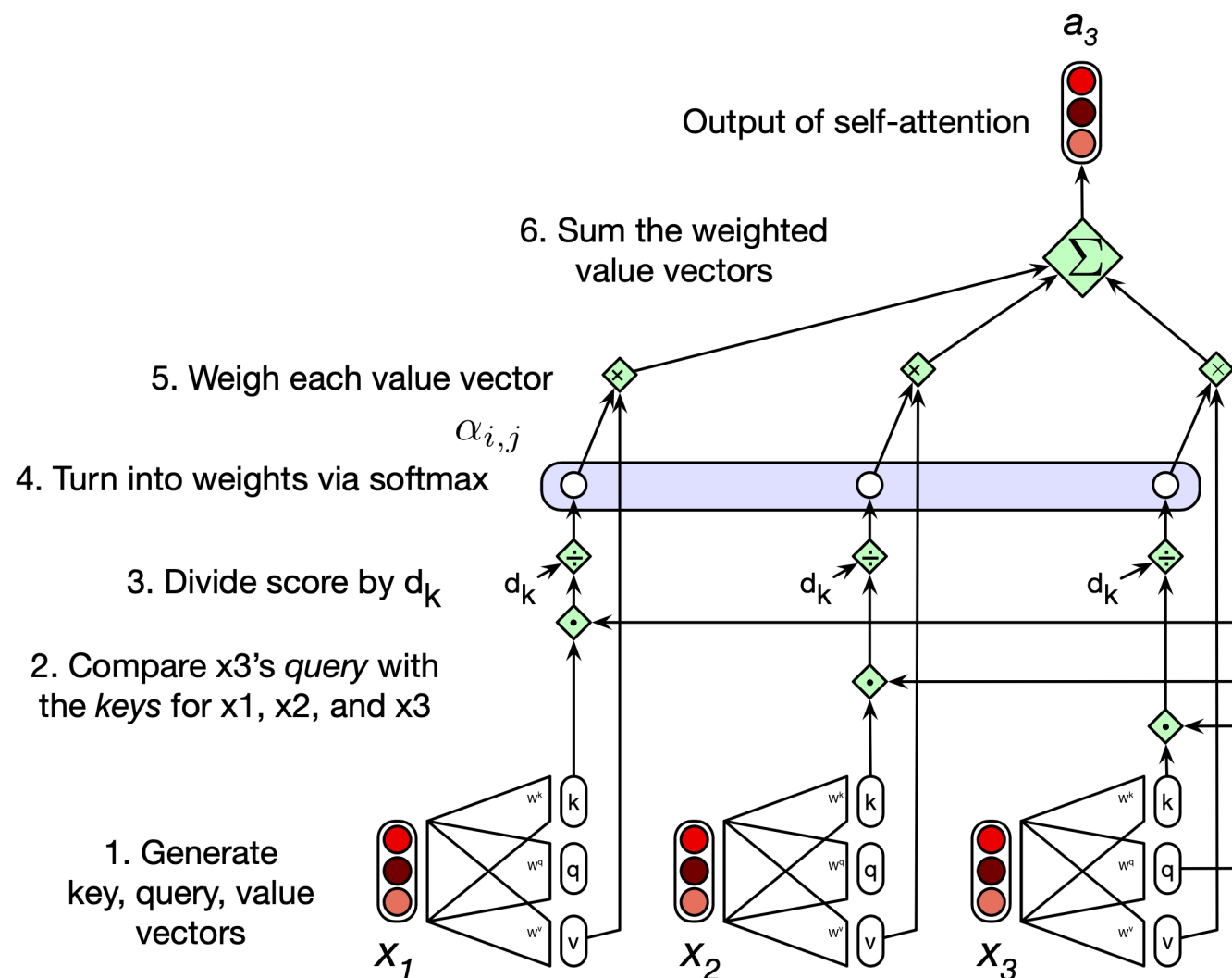
- To fix numeric issues in exponentiating large values, scale down the dot-product by the size of embedding:

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Self-attention example



Self-Attention Final Version



$$\begin{aligned} q_i &= x_i W^Q \\ k_i &= x_i W^K \\ v_i &= x_i W^V \end{aligned}$$

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

$$\forall j \leq i$$

$$a_i = \sum_{j \leq i} \alpha_{ij} v_j$$

Parallelizing Self-Attention using Matrices

- Given a sequence of N input tokens, then the input embedding is $\mathbf{X} \in \mathbb{R}^{N \times d}$ (N could be 512, 1024, 2048, 4096, or more)
- Compute the query, key, and value vectors for N tokens all at once:

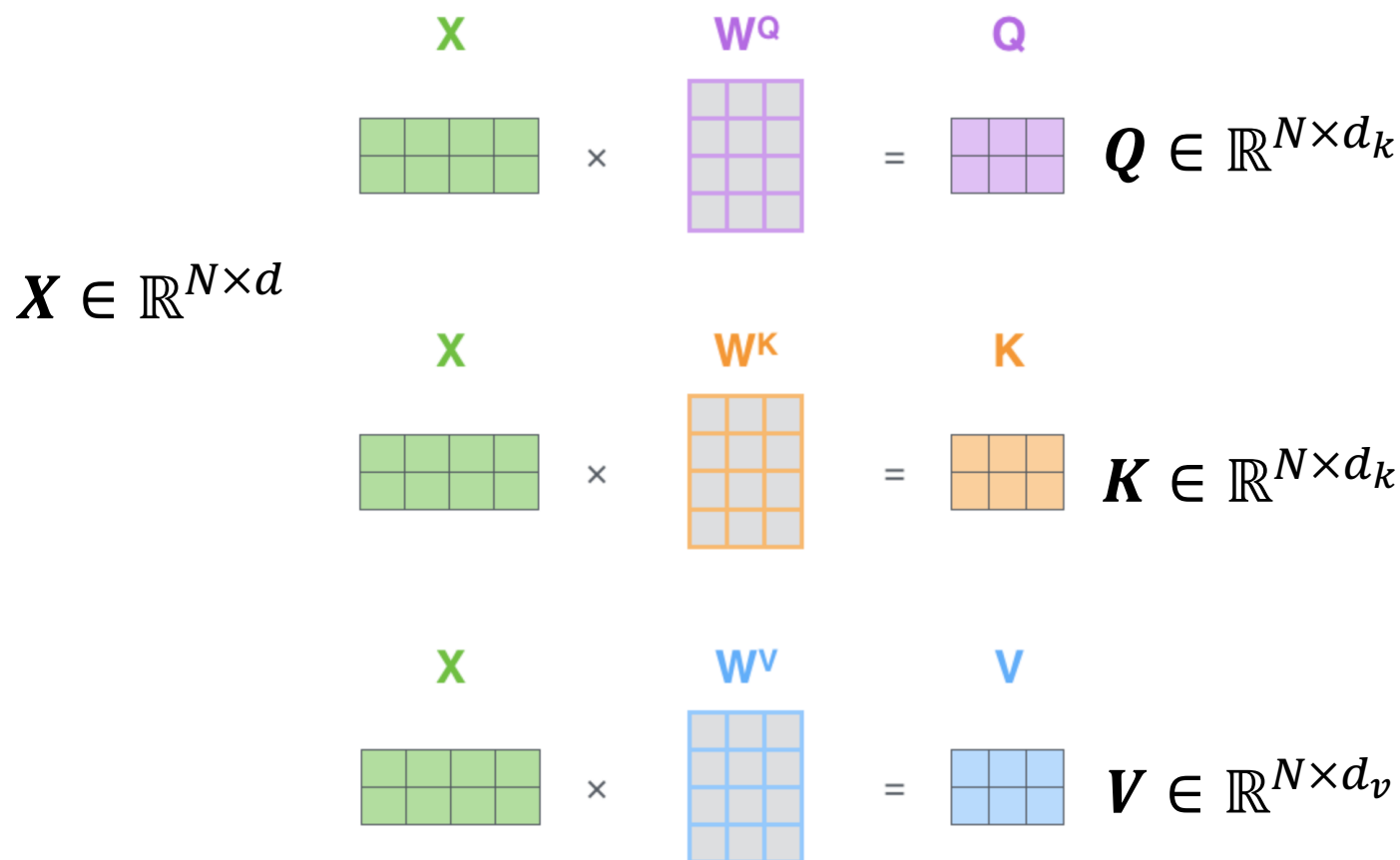
$$\mathbf{Q} = \mathbf{XW}^Q; \quad \mathbf{K} = \mathbf{XW}^K; \quad \mathbf{V} = \mathbf{XW}^V$$

- Resulting in three matrices: $\mathbf{Q} \in \mathbb{R}^{N \times d_k}$, $\mathbf{K} \in \mathbb{R}^{N \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{N \times d_v}$
- Then the attention score can be computed by $\mathbf{QK}^T$, resulting in $N \times N$ scores
- The output:

$$\mathbf{A} = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

Note two issues: 1) $\mathbf{QK}^T$ results in a $N \times N$ score matrix, in which the future needs be masked
2) $\mathbf{A} \in \mathbb{R}^{N \times d}$ and $\mathbf{V} \in \mathbb{R}^{N \times d_v}$, if $d \neq d_v$, then need concatenation (multihead)

Parallelizing Self-Attention using Matrices



$$\text{softmax}\left(\frac{\mathbf{Q} \times \mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

$= \mathbf{Z} \in \mathbb{R}^{N \times d_v}$

Source: <https://jalamar.github.io/illustrated-transformer/>

Masking Out the Future

- Problem of self-attention: QK^T results in a score for each query to every key, including those that come after the query
- Cheating -- Guessing the next word that is already known
- **Fix:** Zero out the upper-triangular portion (set to $-\infty$, which softmax will turn to 0), thus eliminating any knowledge of words from the future

	N				
$q1 \cdot k1$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	
$q2 \cdot k1$	$q2 \cdot k2$	$-\infty$	$-\infty$	$-\infty$	
$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$-\infty$	$-\infty$	
$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$	$-\infty$	N
$q5 \cdot k1$	$q5 \cdot k2$	$q5 \cdot k3$	$q5 \cdot k4$	$q5 \cdot k5$	

Multihead Attention

- **Motivation:** Different words relate to other words in the sentence in many different ways simultaneously
- Ex. *syntactic*, *semantic*, and *discourse* relationships between verb and its argument
- **Difficult** for a single self-attention model to capture all these relations
- **Solution: multihead self-attention layers**
 - Parallel layers at the same depth; each with its own parameters
- Each head, i , has its own query, key, and value matrices:
 $\mathbf{W}_i^Q \in \mathbb{R}^{N \times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{N \times d_k}$, and $\mathbf{W}_i^V \in \mathbb{R}^{N \times d_v}$
- Given input $\mathbf{X} \in \mathbb{R}^{N \times d}$, They produce $\mathbf{Q}_i \in \mathbb{R}^{N \times d_k}$, $\mathbf{K}_i \in \mathbb{R}^{N \times d_k}$, and $\mathbf{V}_i \in \mathbb{R}^{N \times d_v}$
- In the original paper, number of heads $h = 8$, $d = 512$, $d_k = d_v = 64$
- The output of each head is of shape $N \times d_v$; the outputs from h heads are concatenated to $N \times h d_v$

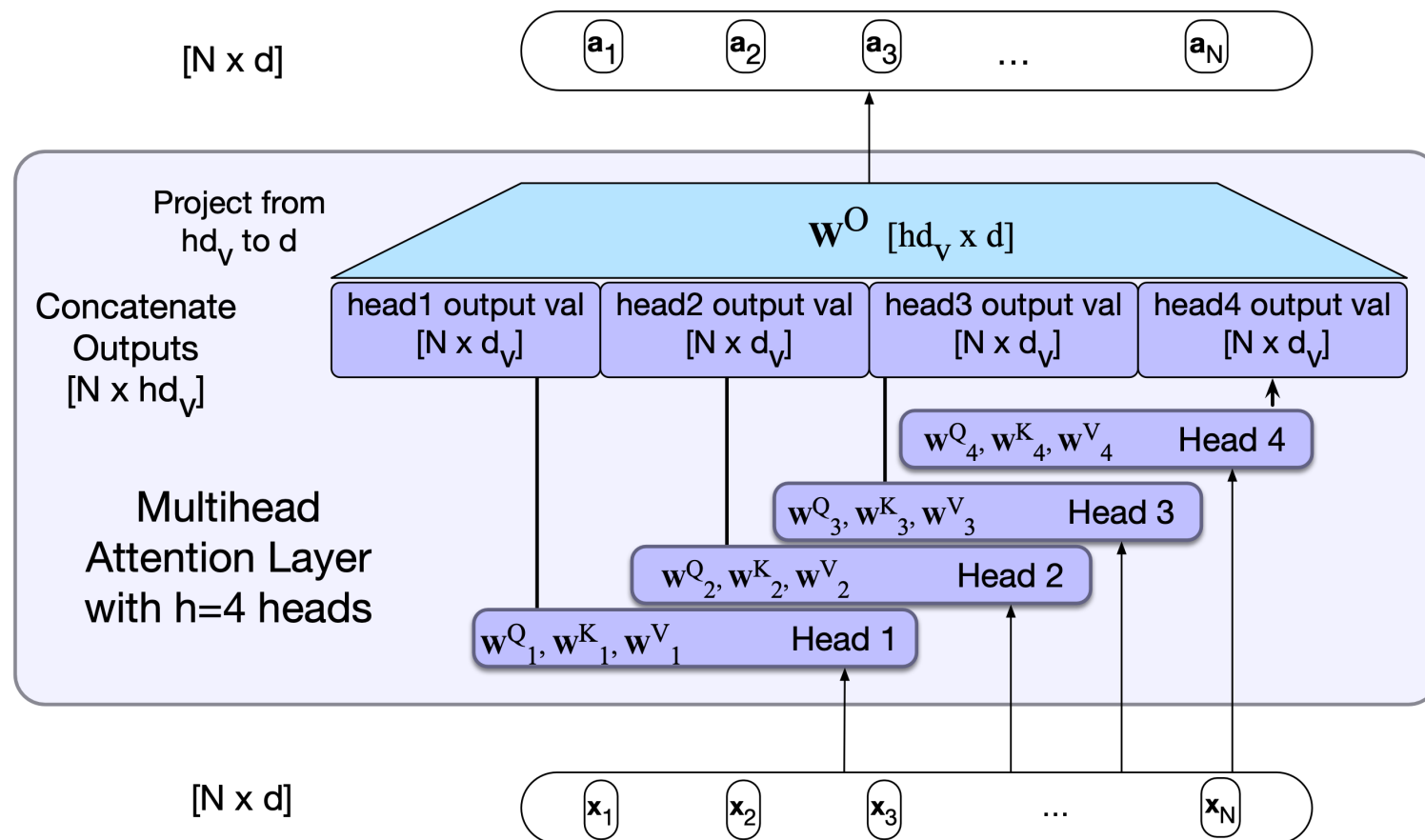
Multihead Attention

- Finally, use yet another linear projection $W^O \in \mathbb{R}^{hd_v \times d}$ to reshape to original dimension d for each token

$$\begin{aligned} Q_i &= XW_i^Q \\ K_i &= XW_i^K \\ V_i &= XW_i^V \end{aligned}$$

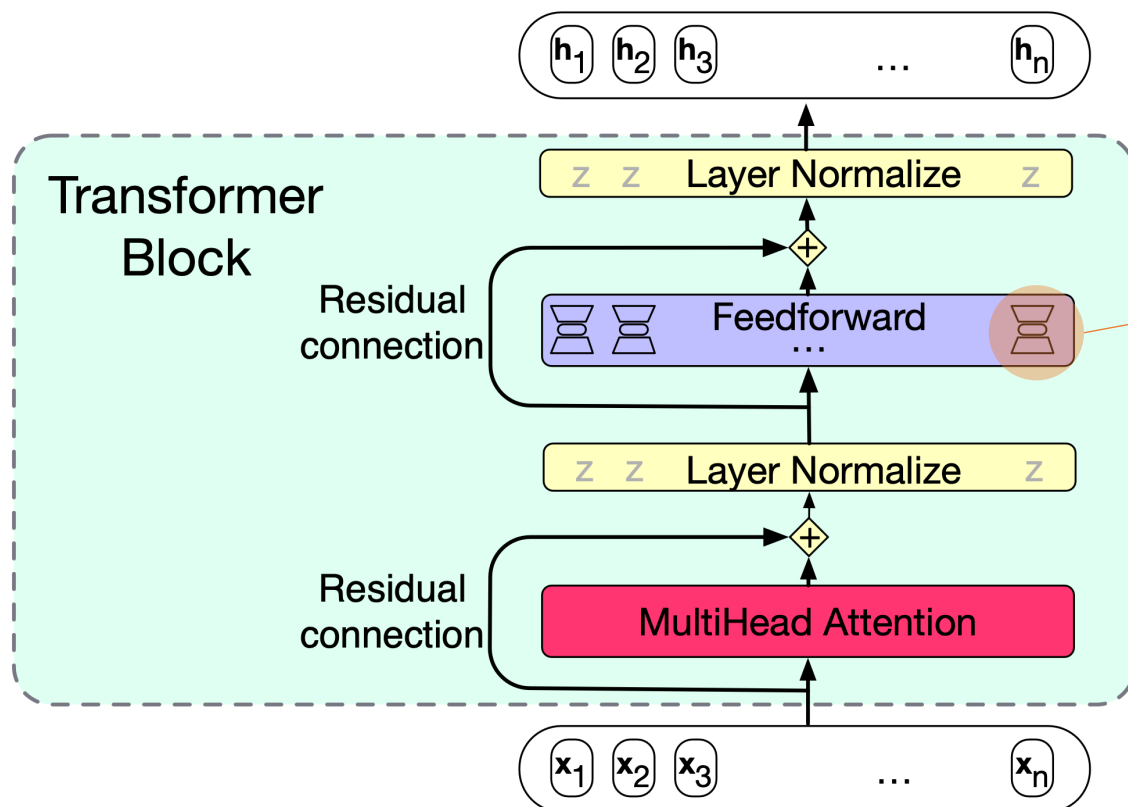
$$\begin{aligned} \text{head}_i &= \text{SelfAttention}(Q_i, K_i, V_i) \end{aligned}$$

$$\begin{aligned} A &= \text{MultiheadAttention}(X) \\ &= (\text{head}_1 \oplus \text{head}_2 \dots \oplus \text{head}_h)W^O \end{aligned}$$



Transformer Blocks

- A Transformer block = Self-attention layer +
- 1) Feedforward layer + 2) Residual connections + 3) Layer normalization

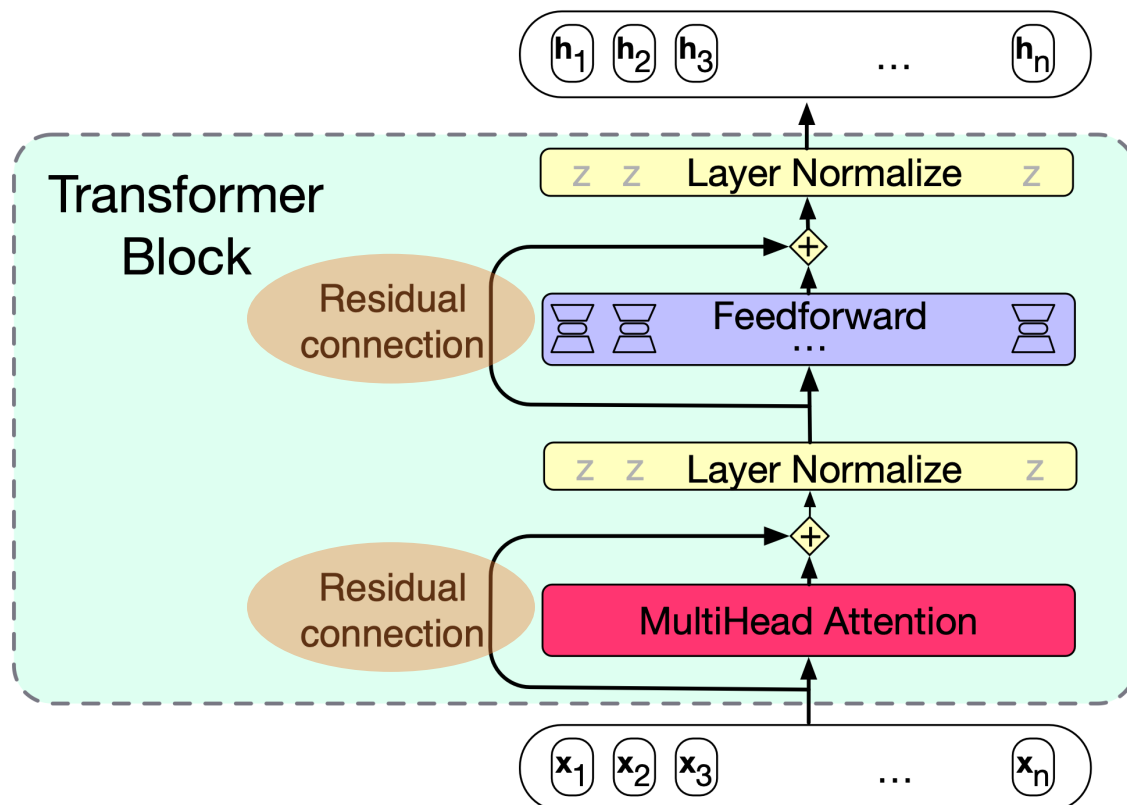


Feedforward layer contains N position-wise networks:

- Each is a 2-layer fully-connected net
- Parameters are different from layer to layer
- Same weights applied to each token position at the same layer
- $d = 512, d_{ff} = 2048$

Transformer Blocks: Residual Connections

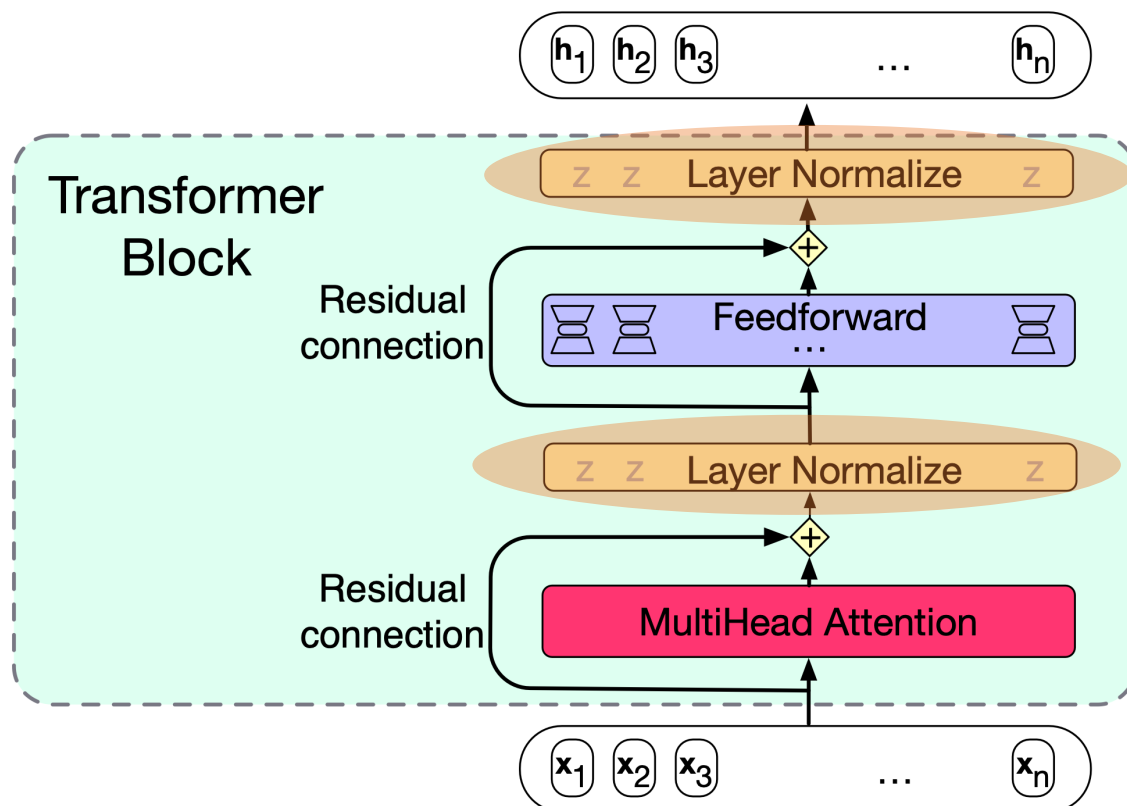
- **Residual connections** pass information from lower layer to higher layer without going through the intermediate layer



- Implemented by adding a layer's input to its output before passing it forward
- Allowing activations to directly go forward and gradients to directly go backward

Transformer Blocks: Layer Normalization

- **Layer normalization** (Ba et al., 2016): keep the values of hidden layer in range that facilitates gradient descent



Input: the vector for a particular position, x
Output: the vector normalized, $\hat{x}$

First, compute the mean and standard deviation over the elements of x :

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

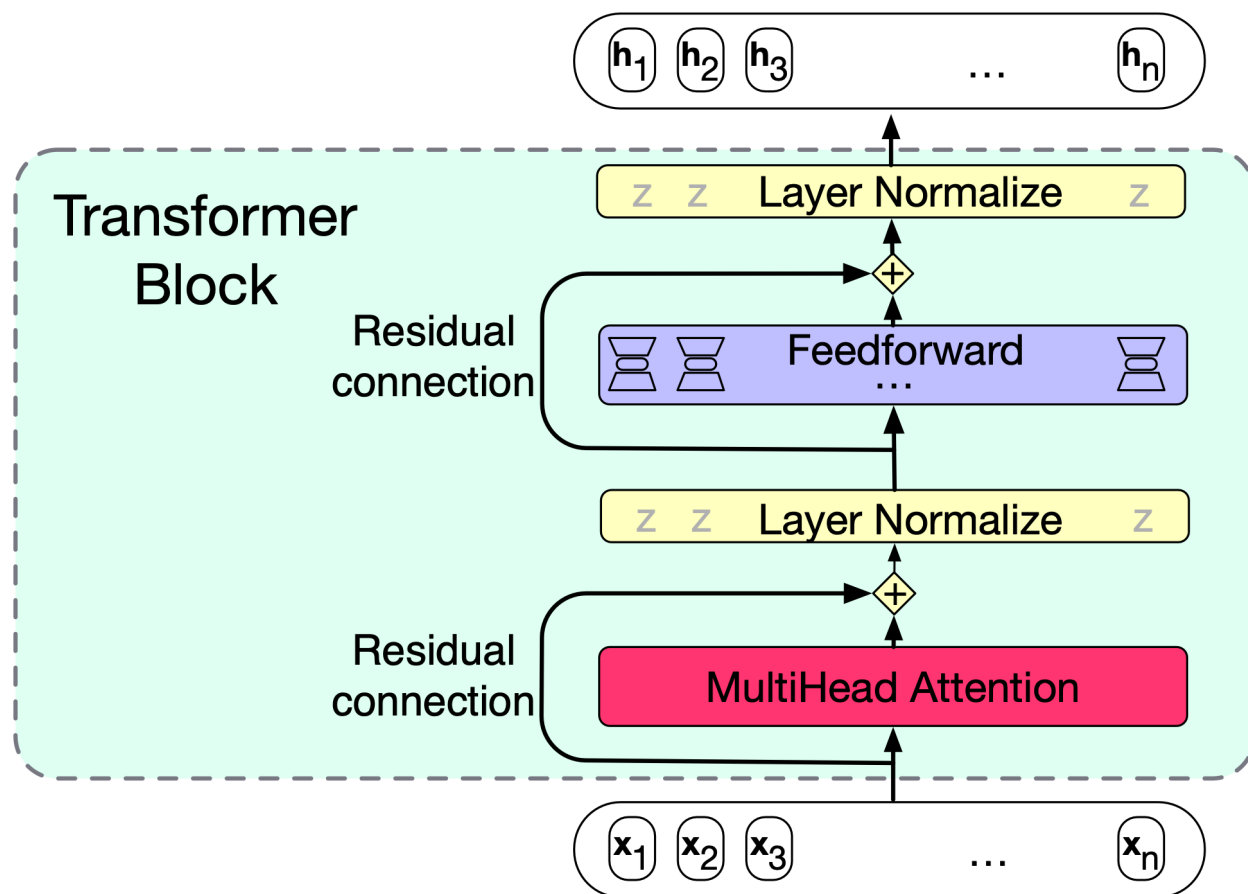
Then, normalize by subtracting and dividing:

$$\hat{x} = (x - \mu) / \sigma$$

Finally, add learnable parameters γ, β

$$\text{LayerNorm} = \gamma \hat{x} + \beta$$

Transformer Blocks: All Together



Input: X , output: H

$$O = \text{LayerNorm}(X + \text{SelfAttention}(X))$$

$$H = \text{LayerNorm}(O + \text{FFN}(O))$$

break down:

Output H

$$H = \text{LayerNorm}(T^5)$$

$$T^5 = T^4 + T^3$$

$$T^4 = \text{FFN}(T^3)$$

$$T^3 = \text{LayerNorm}(T^2)$$

$$T^2 = X + T^1$$

$$T^1 = \text{SelfAttention}(X)$$

Input X

Transformer Blocks

- A bit difference between encoder and decoder

For encoder:

$$X'_l = \text{LayerNorm}(\text{MHA}(X_{l-1}) + X_{l-1}),$$

$$X_l = \text{LayerNorm}(\text{FFN}(X'_l) + X'_l),$$

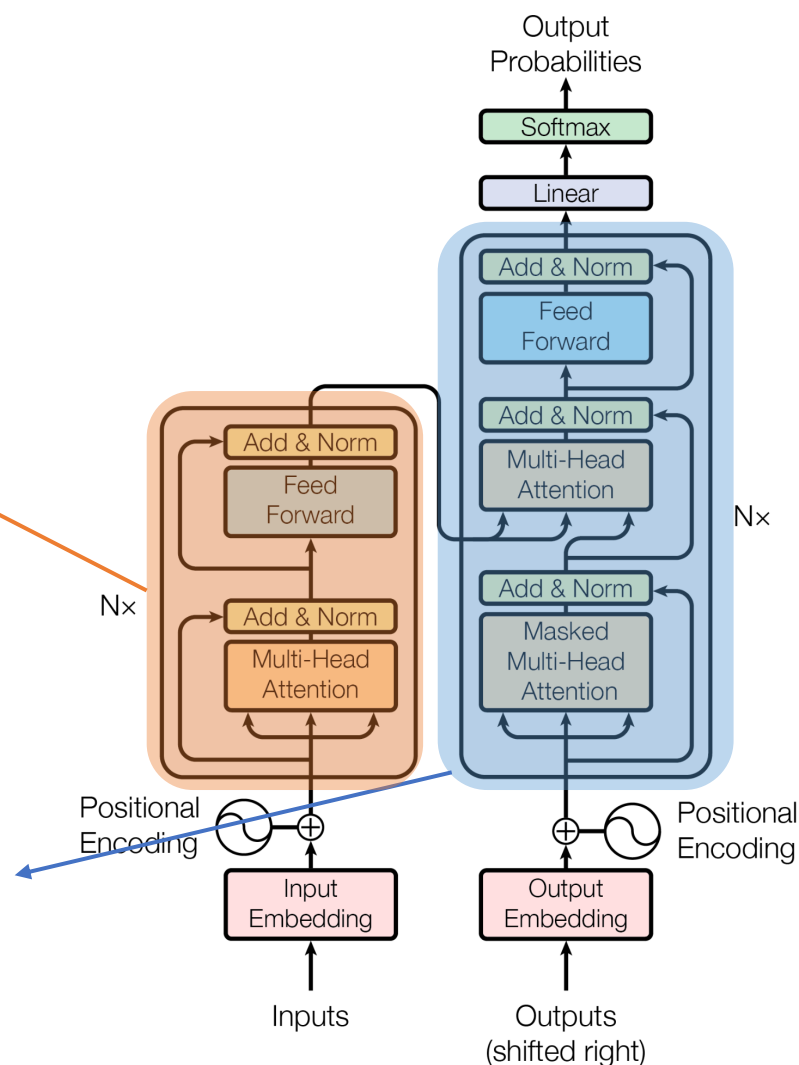
(“MHA” for **M**ulti-**H**ead Self-**A**ttention)

For decoder:

$$Y'_l = \text{LayerNorm}(\text{MaskedMHA}(Y_{l-1}) + Y_{l-1}),$$

$$Y''_l = \text{LayerNorm}(\text{CrossMHA}(Y'_l, X_L) + Y'_l),$$

$$Y_l = \text{LayerNorm}(\text{FFN}(Y''_l) + Y''_l),$$



Overview

第五周看到这里

- Motivation
- Transformer Basics
 - Self-Attention; Multihead Attention
 - Transformer Blocks
 - **Positional Embedding**
- Transformer Details
- Extensions

Input Embeddings for Token and Position

- Input embedding to a transformer block $\mathbf{X}$: a matrix of shape $N \times d$
- It has two components: **token** embedding + **positional** embedding

Token embedding:
One-hot vectors $\Rightarrow$
dense vectors

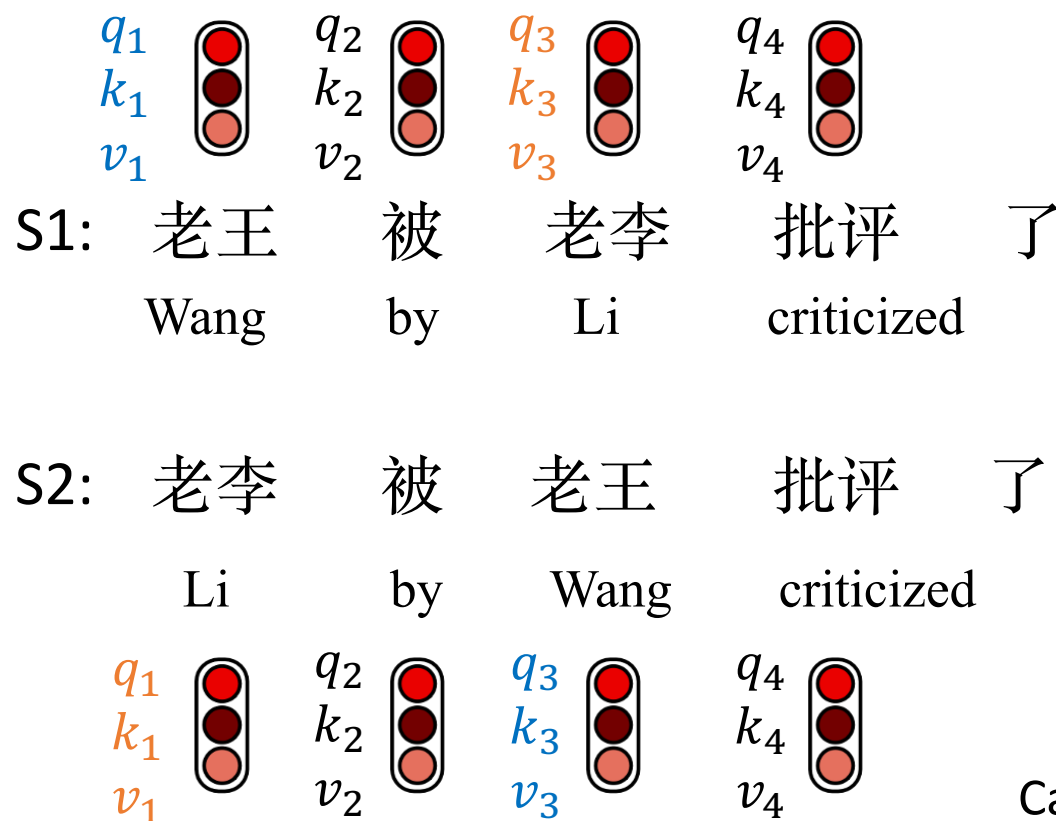
$$\begin{array}{c} 1 \text{ token} \\ 1 \end{array} \begin{array}{c} 5 \\ |V| \end{array} \begin{array}{|c|} \hline 0000100 \dots 0000 \\ \hline \end{array} \times \begin{array}{c} d \\ 5 \\ |V| \end{array} \begin{array}{|c|} \hline \text{E} \\ \hline \end{array} = 1 \begin{array}{c} d \end{array}$$

Problem: token
embeddings are not
position-dependent

$$\begin{array}{c} N \text{ tokens} \\ N \end{array} \begin{array}{c} |V| \\ \begin{array}{|c|} \hline 0000100 \dots 0000 \\ \hline 0000000 \dots 0010 \\ \hline 1000000 \dots 0000 \\ \hline \dots \\ \hline 0000100 \dots 0000 \\ \hline \end{array} \end{array} \times \begin{array}{c} d \\ |V| \end{array} \begin{array}{|c|} \hline \text{E} \\ \hline \end{array} = \begin{array}{c} d \\ N \end{array} \begin{array}{|c|} \hline \text{token embeddings} \\ \hline \end{array}$$

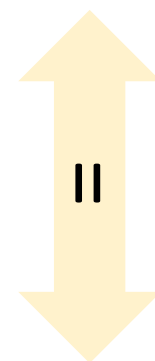
Necessity of Positional Embedding

- Word order matters



representation for “批评” in two sentences

$$a_4 = \alpha_{41}v_1 + \alpha_{42}v_2 + \alpha_{43}v_3$$



Exactly the same if
“老王” and “老李” are
represented by token
embeddings only

$$a_4 = \alpha_{41}v_1 + \alpha_{42}v_2 + \alpha_{43}v_3$$

Cannot tell who criticized whom by looking at the output

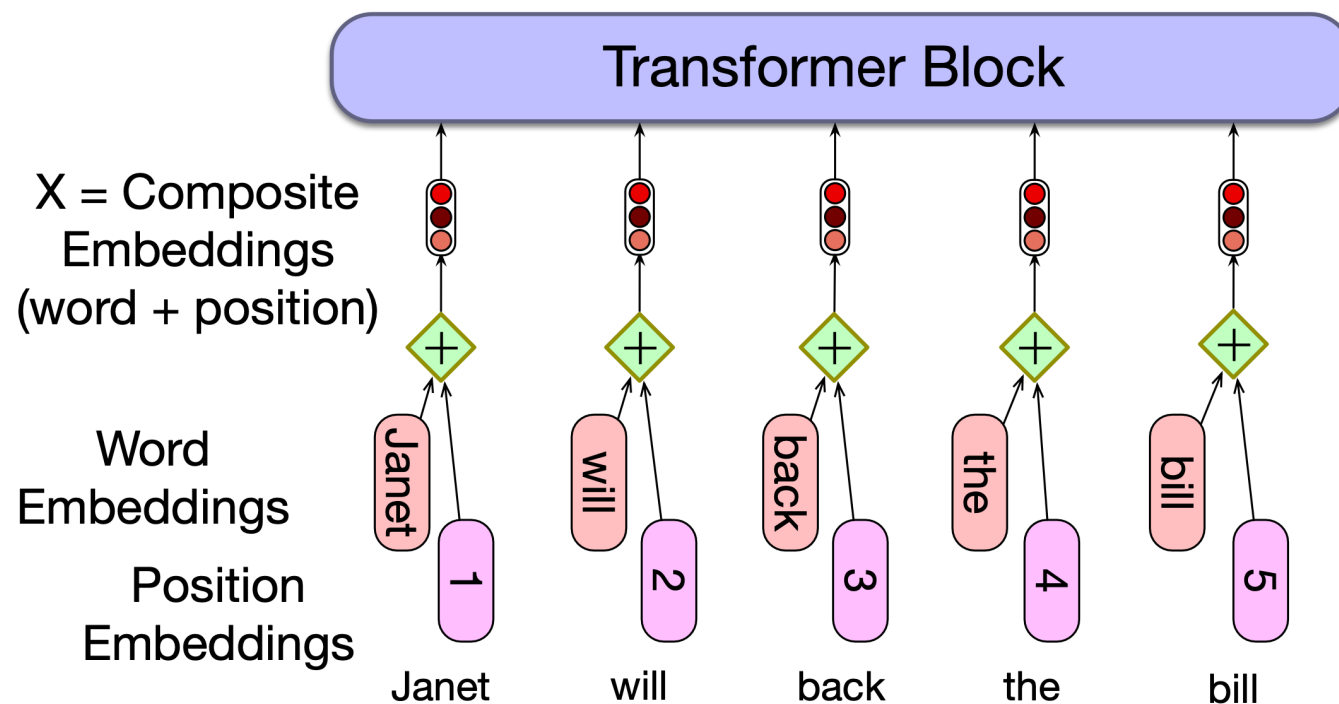
Positional Embedding

- Simplest method: **learned** absolute position (used in GPT-2)
- Randomly initialize embeddings for each possible input position

$$X_{Janet} = E_{ID(Janet)} + P_i$$

E_i and P_i are of same dimension d

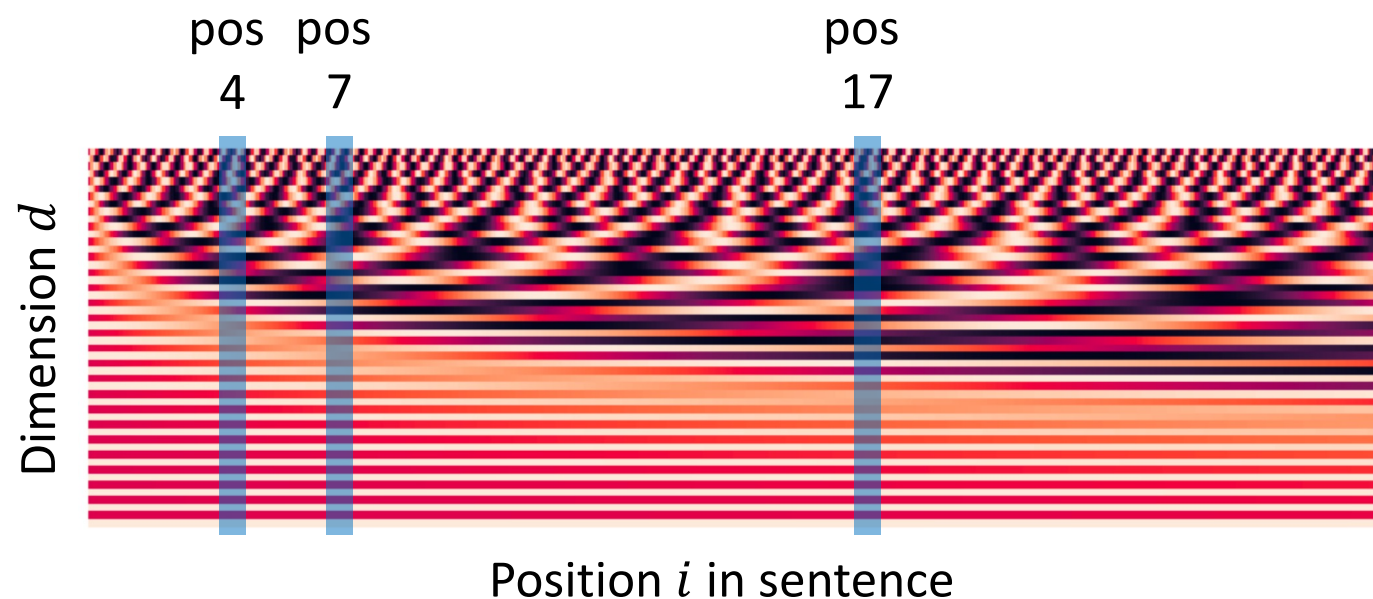
Drawback: Initial positions (small values) dominate the training data; later positions (large values) are poorly trained



Absolute Positional Embedding

- **Sinusoidal (正弦曲线) positional embedding** by concatenating sine and cosine functions of varying periods (First used in the 2017 Attention paper)

$$P_i = \begin{bmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \sin(i/10000^{2*2/d}) \\ \cos(i/10000^{2*2/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{bmatrix} \quad \left. \vphantom{\begin{bmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \sin(i/10000^{2*2/d}) \\ \cos(i/10000^{2*2/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{bmatrix}} \right\} d \quad (= 512)$$



pos 4 is “closer” to pos 7 than pos 17

Alternative: Rotary Position Embedding (RoPE)

- Construct a rotation matrix to encode relative positions (Su et al., 2021)

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad \begin{array}{l} \text{Here, } m \text{ is the} \\ \text{index of input} \\ \text{embedding} \\ \mathbf{x}_m \end{array}$$

Position i : $\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q \mathbf{R}_{\theta, i}$, position j : $\mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K \mathbf{R}_{\theta, j}$

Then attention $A_{ij} = \mathbf{q}_i \mathbf{k}_j^\top = \mathbf{x}_i \mathbf{W}^Q \mathbf{R}_{\theta, i} (\mathbf{x}_j \mathbf{W}^K \mathbf{R}_{\theta, j})^\top = \mathbf{x}_i \mathbf{W}^Q \mathbf{R}_{\theta, i} \mathbf{R}_{\theta, j}^\top \mathbf{W}^{K\top} \mathbf{x}_j^\top$

Rotary Position Embedding (RoPE) cont.

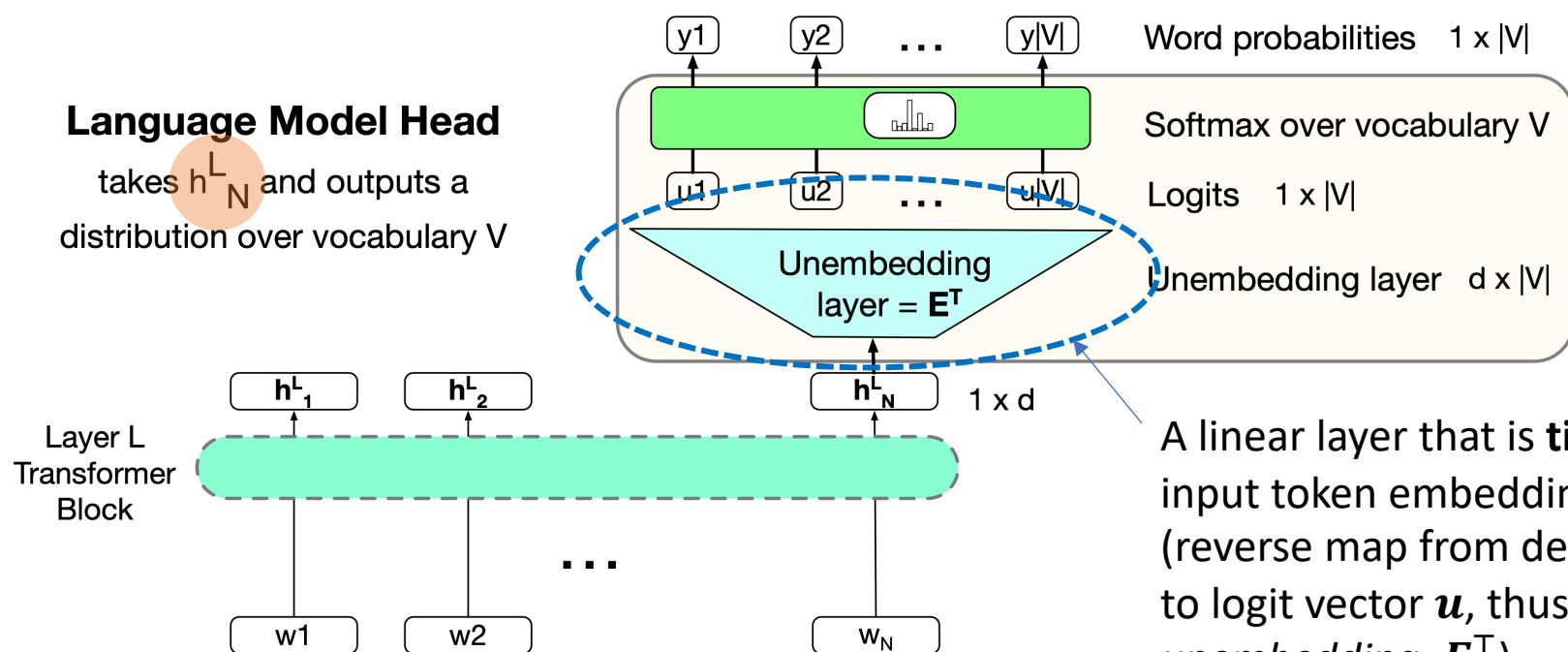
- $A_{ij} = \mathbf{q}_i \mathbf{k}_j^\top = \mathbf{x}_i \mathbf{W}^Q \mathbf{R}_{\theta,i} \mathbf{R}_{\theta,j}^\top \mathbf{W}^{K\top} \mathbf{x}_j^\top = \mathbf{x}_i \mathbf{W}^Q \mathbf{R}_{\theta,i-j} \mathbf{W}^{K\top} \mathbf{x}_j^\top$
- Attention is modeled by the rotation matrix $\mathbf{R}_{\theta,i-j}$, which includes relative positional information
- **Advantages** of RoPE:
 - Dot product between $\mathbf{q}_i$ and $\mathbf{k}_j^\top$ depend only on relative distance, allowing the model to understand ordering.
 - Generalize better to longer context lengths.
 - Multiplication can be efficiently implemented.

Overview

- Motivation
- Transformer Basics
- **Transformer Details**
 - Language Modeling Head
 - Architectures
 - Training
- Extensions

Language Modeling Head

- **Head**: additional neural network added on top of the basic transformer architecture to enable certain task
- LM head $\Rightarrow$ Language Modeling task: Predicting next word from **context**



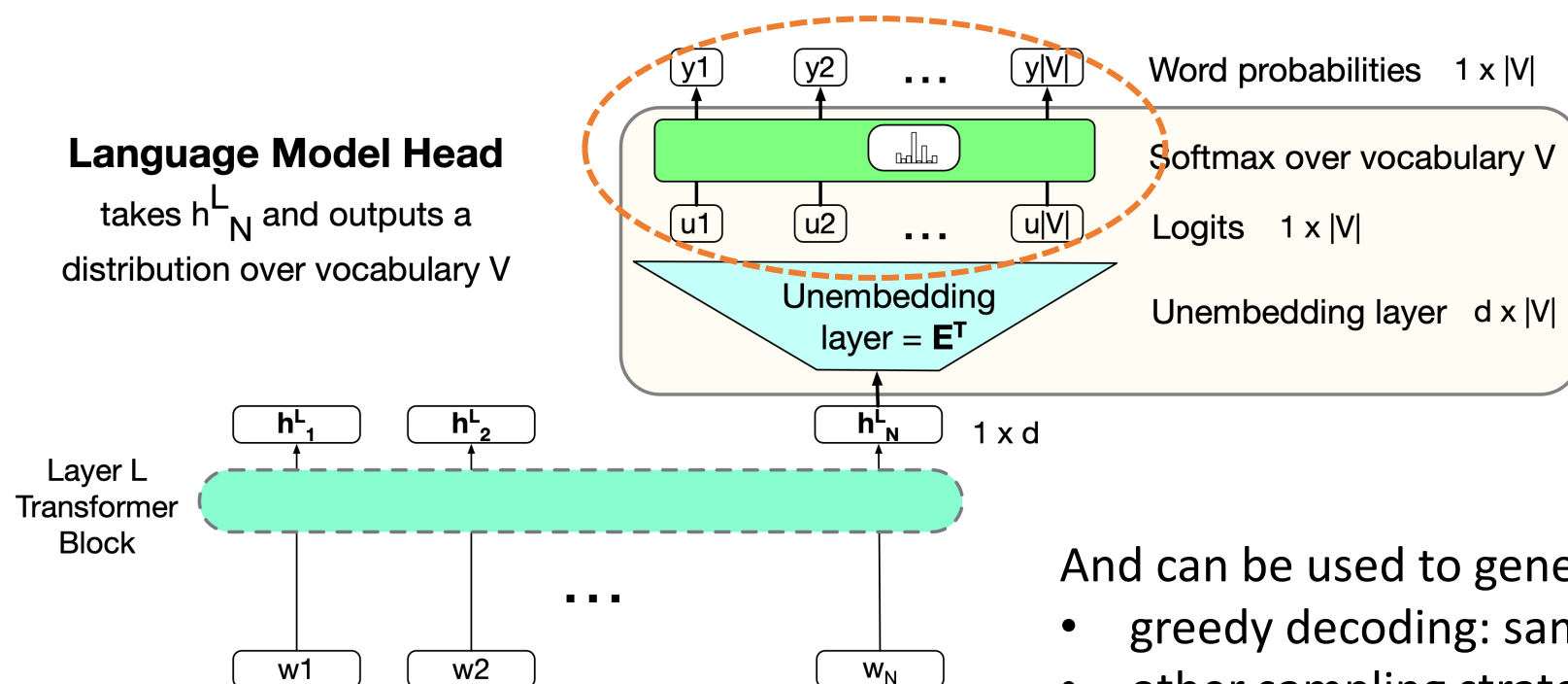
Take the output of the last token position N from the last layer L , h_N^L , to predict the next word

A linear layer that is **tied** to the input token embedding matrix E (reverse map from dense vector h to logit vector u , thus called *unembedding*, E^T)

$$u = h_N^L E^T$$

Language Modeling Head

- A softmax layer turns the logits $\mathbf{u}$ into the probabilities $\mathbf{y}$ over vocabulary



$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

After being trained, these $\mathbf{y}$ distribution can be used to estimate the probability of a given text

- And can be used to generate text, by sampling from $\mathbf{y}$
- greedy decoding: sample the highest prob word
 - other sampling strategies: top-k, top-p, temperature

Training Transformer as Language Model

- Just like training an RNN language model, we use the same **self-supervision** training method: **predict the next word**
- At each position t , take as input the word sequence $w_{1:t}$, and use them to compute a probability distribution $\hat{\mathbf{y}}_t$, and then compute the **cross-entropy** loss between this distribution and the actual next word $\mathbf{y}_t[w_{t+1}]$

$\mathbf{y}_t$ is a one-hot vector of length $|V|$:

- $\mathbf{y}_t[w = w_{t+1}] = 1$
- $\mathbf{y}_t[w \neq w_{t+1}] = 0$

Cross-entropy loss:

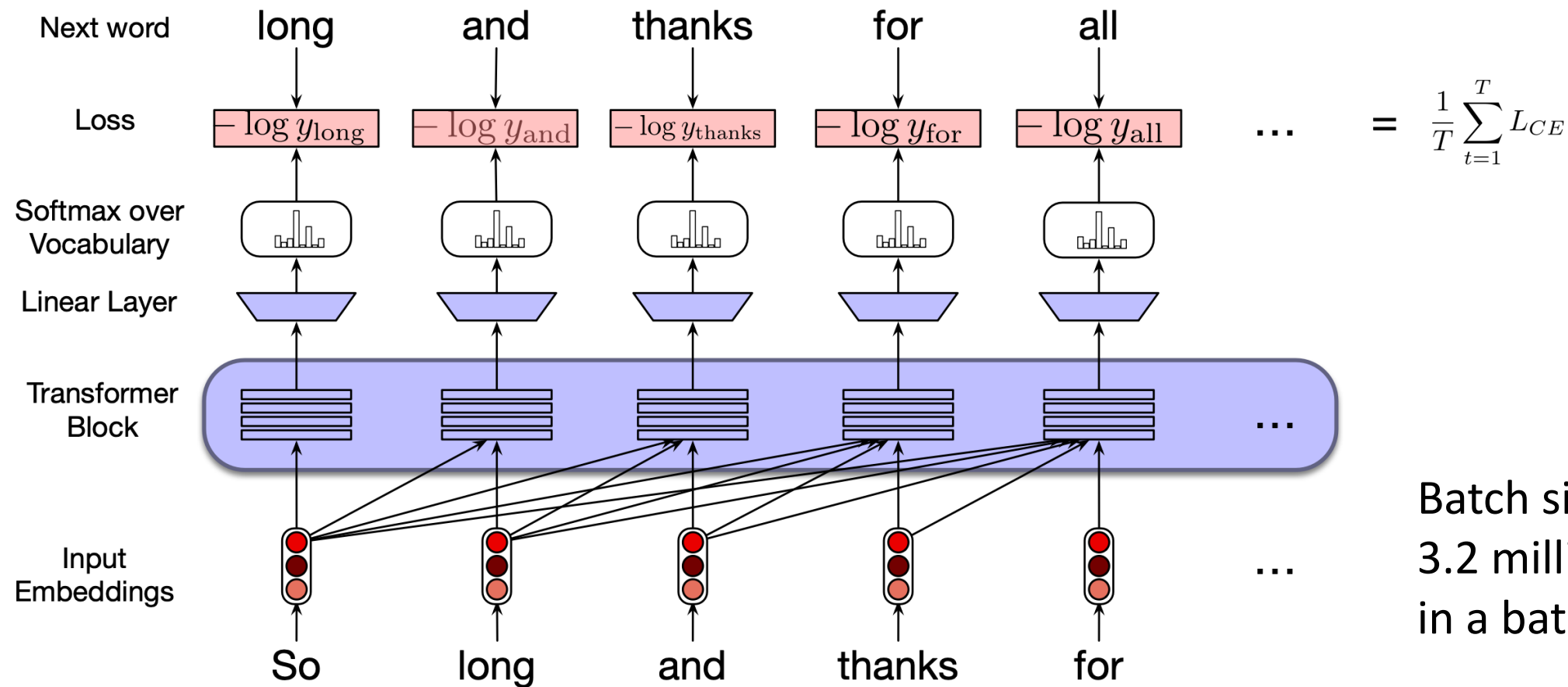
- $L_{CE} = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t = -\log \hat{\mathbf{y}}_t[w_{t+1}]$
- Then average over entire sequence: $\frac{1}{T} \sum_{t=1}^T L_{CE}$

$T = 2048$ or 4096 for GPT3 or GPT3.5

GPT4: 8192; GPT4 Turbo: 128000

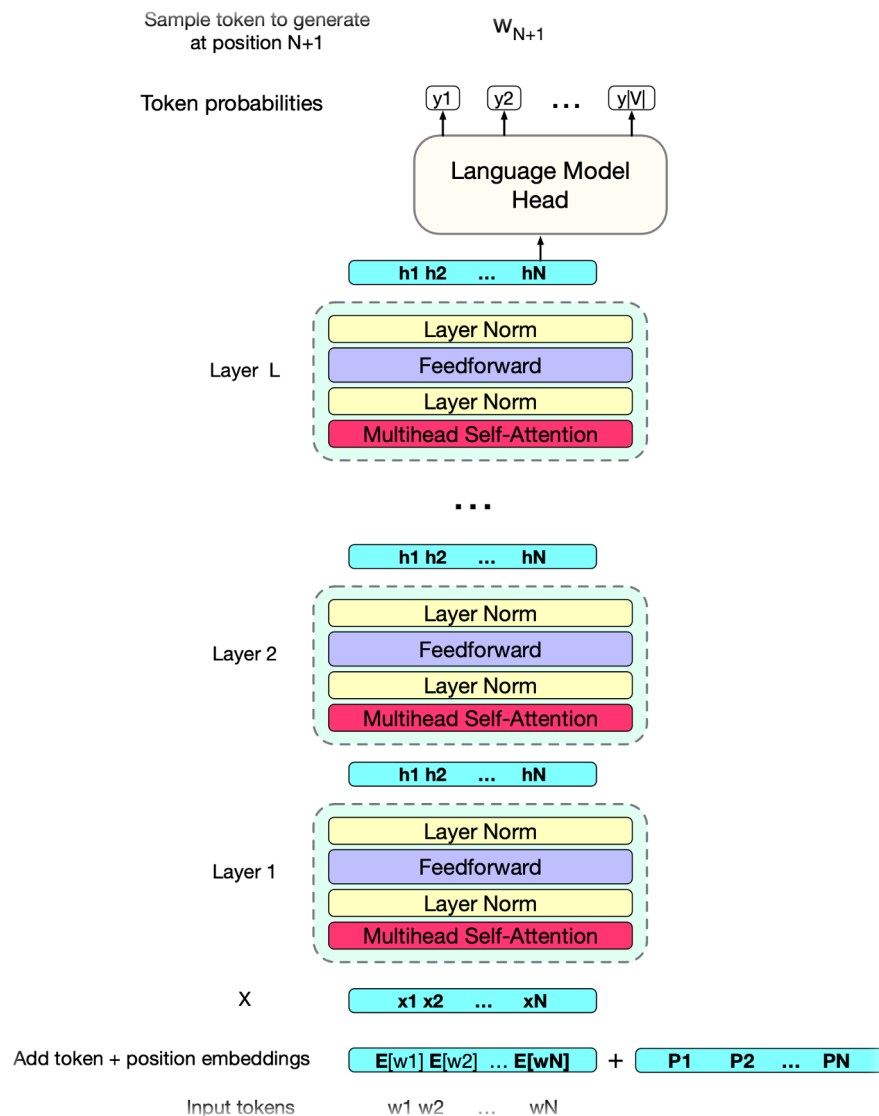
Source: <https://www.scriptbyai.com/token-limit-openai-chatgpt>

Training Transformer as Language Model



Batch size is large:
3.2 million tokens
in a batch for GPT3

Architecture: Decoder-only



Transformer model has a stacked architecture of multiple layers of transformer blocks

The input to the first block is word embedding + positional embedding: $X = E_w + P_i$

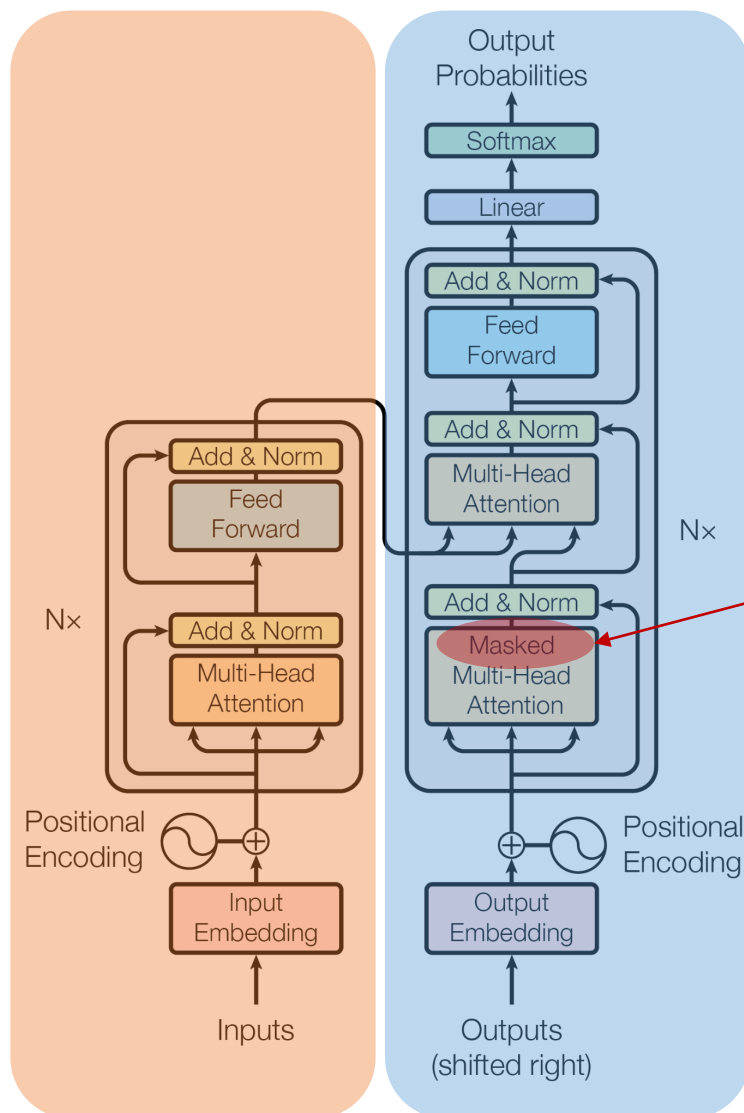
The input to all the other layers is the output H from the layer below

Terminology:

- **decoder-only model**: it is only half of the encoder-decoder model in the original work

Architecture: Encoder-decoder

Encoder



Decoder

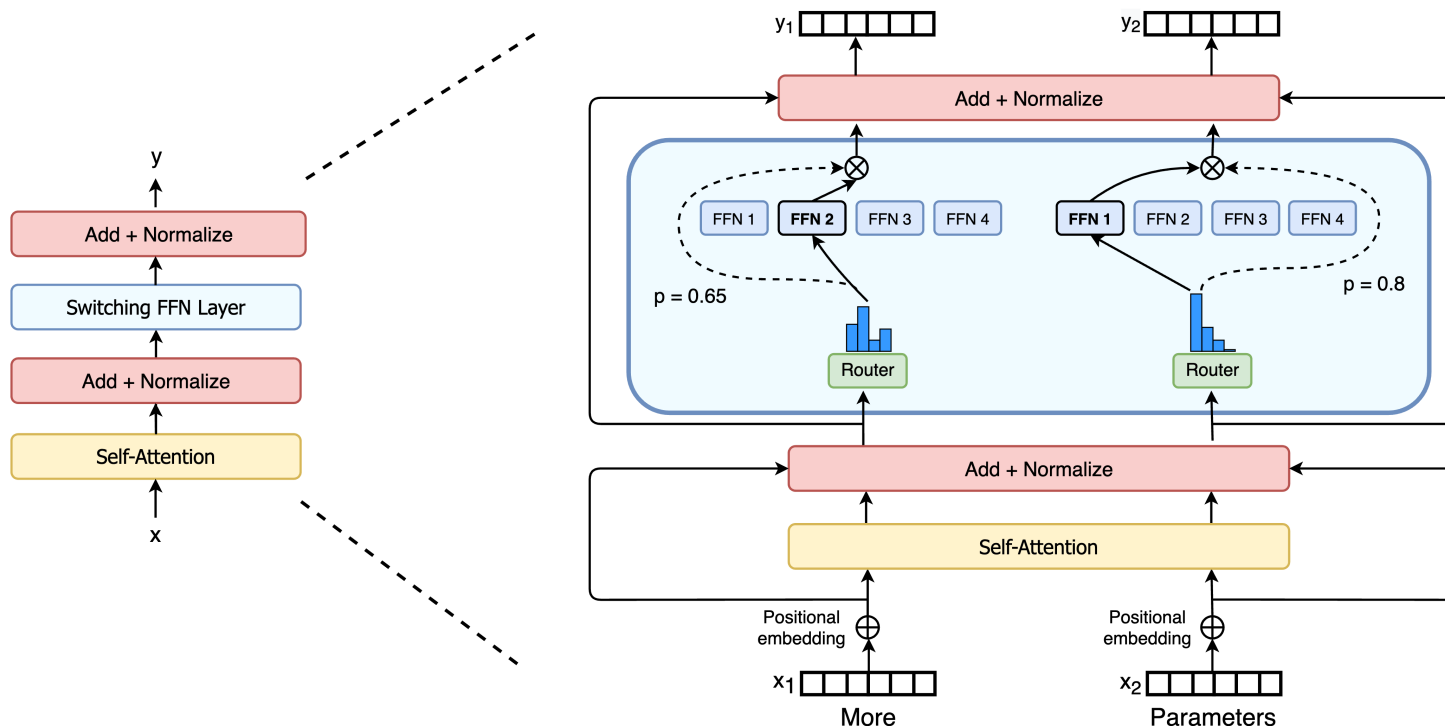
Vaswani et al. (2017)'s original work is for machine translation task

- In the encoder, the self-attention is bidirectional;
- In the decoder, the self-attention is causal, i.e., future words are masked out

It was only later that the paradigm for causal language model was defined using only the decoder part

Architecture: Mixture of Experts (MoE)

- How to train faster? -- Replace every FFN layer of the transformer model with an MoE layer



Each MoE layer has K experts:

$$E_1, E_2, \dots, E_K$$

Each E_i is a FFN

Each token x_t is passed through a **router**, or **gate** G , computing the weight of each expert

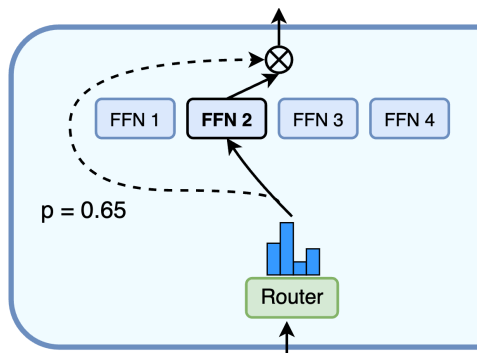
$$G(x_t) = \text{softmax}(\text{topk}(x_t \cdot W^G))$$

Architecture: Mixture of Experts (MoE) cont.

- Only top k experts will be selected (other non-selected experts will be placed with weight 0)

$$G(\mathbf{x}_t) = \text{softmax}(\text{topk}(\mathbf{x}_t \cdot \mathbf{W}^G))$$

- $G(\mathbf{x}_t)$ is in the form: $[G(\mathbf{x}_t)_1, \dots, G(\mathbf{x}_t)_k]$
- The output is the weighted sum of all selected experts



$$\text{MoELayer}(\mathbf{x}_t) = \sum_{i=1}^K G(\mathbf{x}_t)_i \cdot E_i(\mathbf{x}_t)$$

Comparison of Model Architecture

	LLaMA-3.1 (405B)	DeepSeek (67B)	DS-V2 (236B)	DS-V3 (671B)
MoE	No	No	162 experts	257 expert
# of layers	126	95	60	61
Hidden dimension d	16384	8192	5120	7168
# of attention heads	128	64	128	128

Types of Normalization

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

- Original transformer: Layer normalization

$$\text{LayerNorm}(\mathbf{x}) = \gamma \hat{\mathbf{x}} + \beta \quad \hat{\mathbf{x}} = (\mathbf{x} - \mu) / \sigma$$

- More advanced methods:
- Root Mean Square Layer Normalization: **RMSNorm**

$$\text{RMSNorm}(\mathbf{x}) = \gamma \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}$$

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$$

- RMSNorm has better speed and performance
- Used in Gopher, Chinchilla, LLaMA 3.1, DeepSeek

Types of Normalization cont.

- DeepNorm
- Scale the residual connection with some factor α

$$\text{DeepNorm}(\mathbf{x}) = \text{LayerNorm}(\alpha \cdot \mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

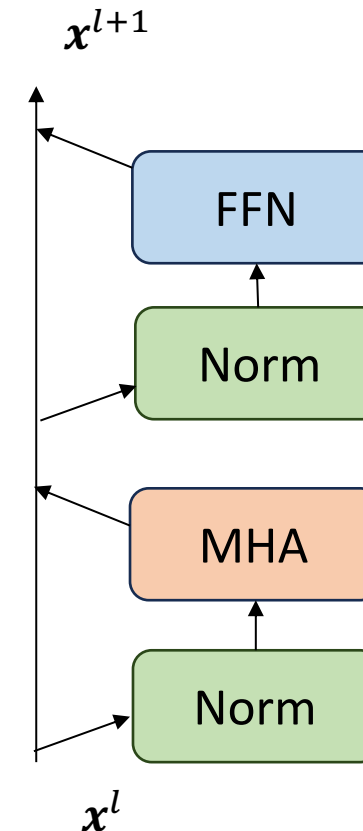
- where Sublayer represents FFN or Self-Attention (MHA)
- More robust training effect, deeper networks (>1000)
- Used in GLM-130B

Position of Normalization

- **Pre-Norm:** normalization placed before residual connection, to every sublayer

$$\text{PreNorm}(\mathbf{x}) = \mathbf{x} + \text{Sublayer}(\text{Norm}(\mathbf{x}))$$

- Pros: Training is more robust
- Cons: Less performant than Post-Norm

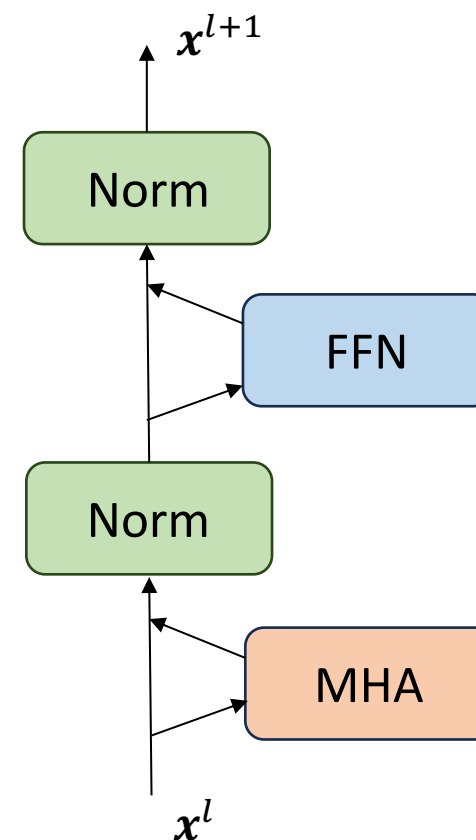


Position of Normalization

- **Post-Norm**: normalization placed after residual connection

$$\text{PostNorm}(\mathbf{x}) = \text{Norm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

- Used in original transformer
- Pros: Training is faster to converge
- Cons: Sometimes makes training less robust



Comparison of Model Architecture cont.

	LLaMA-3.1 (405B)	DeepSeek (67B)	DS-V2 (236B)	DS-V3 (671B)
MoE	No	No	162 experts	257 expert
# of layers	126	95	60	61
Hidden dimension d	16384	8192	5120	7168
# of attention heads	128	64	128	128
Normalization	Pre RMSNorm	Pre RMSNorm	Pre RMSNorm	Pre RMSNorm

Training: Data

- Large Language Models (LLMs) are mainly trained on text scraped from the web, augmented by more carefully curated data
- Such large data are likely to contain many natural examples that are helpful to NLP tasks: question and answer pairs, translation, summaries, etc.
- **Common Crawl**: A series of snapshots of the entire web with billions of webpages
- Needs a LOT of cleaning work!
- Ex. Colossal Clean Crawled Corpus (C4; Raffel et al., 2020): 156 billion tokens
 - Mostly patent text documents, Wikipedia, and news sites (according to Dodge et al., 2021)
- MNBVC(Massive Never-ending BT Vast Chinese corpus) 超大规模中文语料集

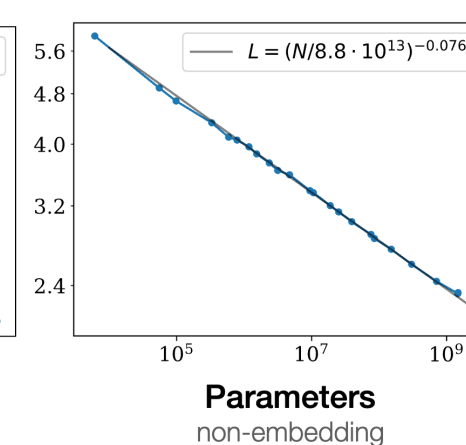
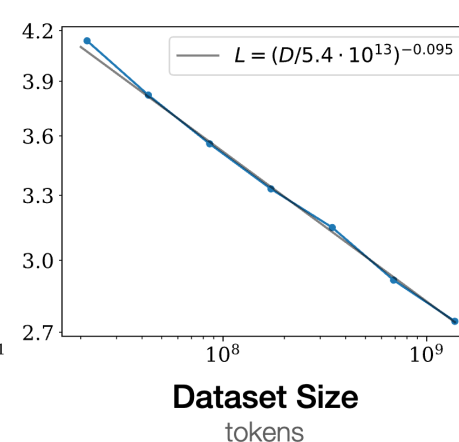
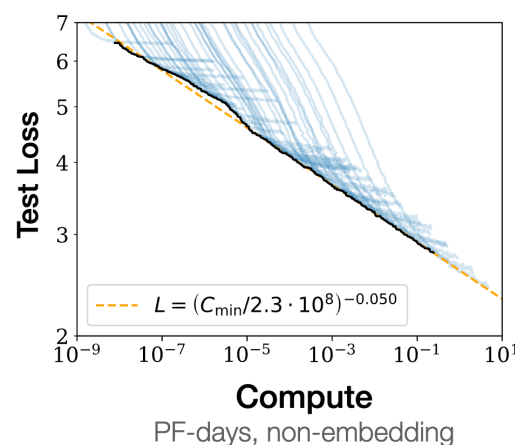
Train: Scaling Laws

- Performance of LLMs are determined by three factors: model size, dataset size, and training time
- Scaling laws:** the model's performance (measured by loss) scales as a *power law* with each of the three factors

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} \quad L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}$$

Loss L as a function of:

- Number of non-embedding parameters N
 - Dataset size D
 - Compute budget C
- (In each case the other two factors are held constant)



Ref: Kaplan et al. (2020) Scaling Laws for Neural Language Models. arxiv.org/abs/2001.08361

Training: Estimate # of Parameters

- The number of non-embedding parameters N can be estimated:
- $N \approx 2d \cdot n_{\text{layer}} \cdot (2d_{\text{attn}} + d_{\text{ff}}) \approx 12n_{\text{layer}}d^2$
- (Assuming $d = d_{\text{attn}} = d_{\text{ff}}$)
- For GPT-3, with $n = 96$ layers and $d = 12288$, $N \approx 12 \times 96 \times 12288^2 \approx 1.74 \times 10^{11}$
- That is 174 billion parameters (千亿)

Overview

- Motivation
- Transformer Basics
- Transformer Details
- **Extensions**

Achievement: Machine Translation

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.0	$2.3 \cdot 10^{19}$	

FLOPS =
Floating point
operations per
second

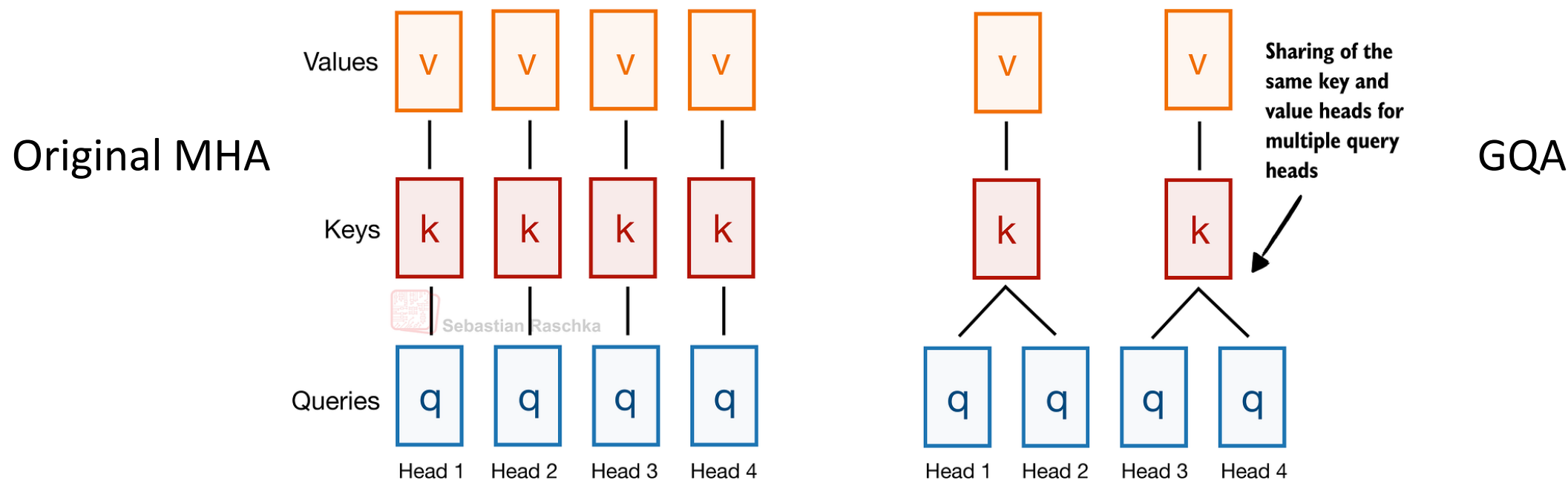
Table from Vaswani et al. (2017)

Drawbacks

- **Quadratic computing time** in self-attention
- Computing all pairs of interactions means that computation grows quadratically with sequence length
- For RNN models, it only grow linearly
- Total number of operation grows in $O(n^2 d)$
- For smaller models, $d \approx 1000$; for larger models, $d > 10000$
- For short input, $n \approx 512$
- For long input (e.g., long documents), $n > 50000$

Improvements to Attention Mechanism

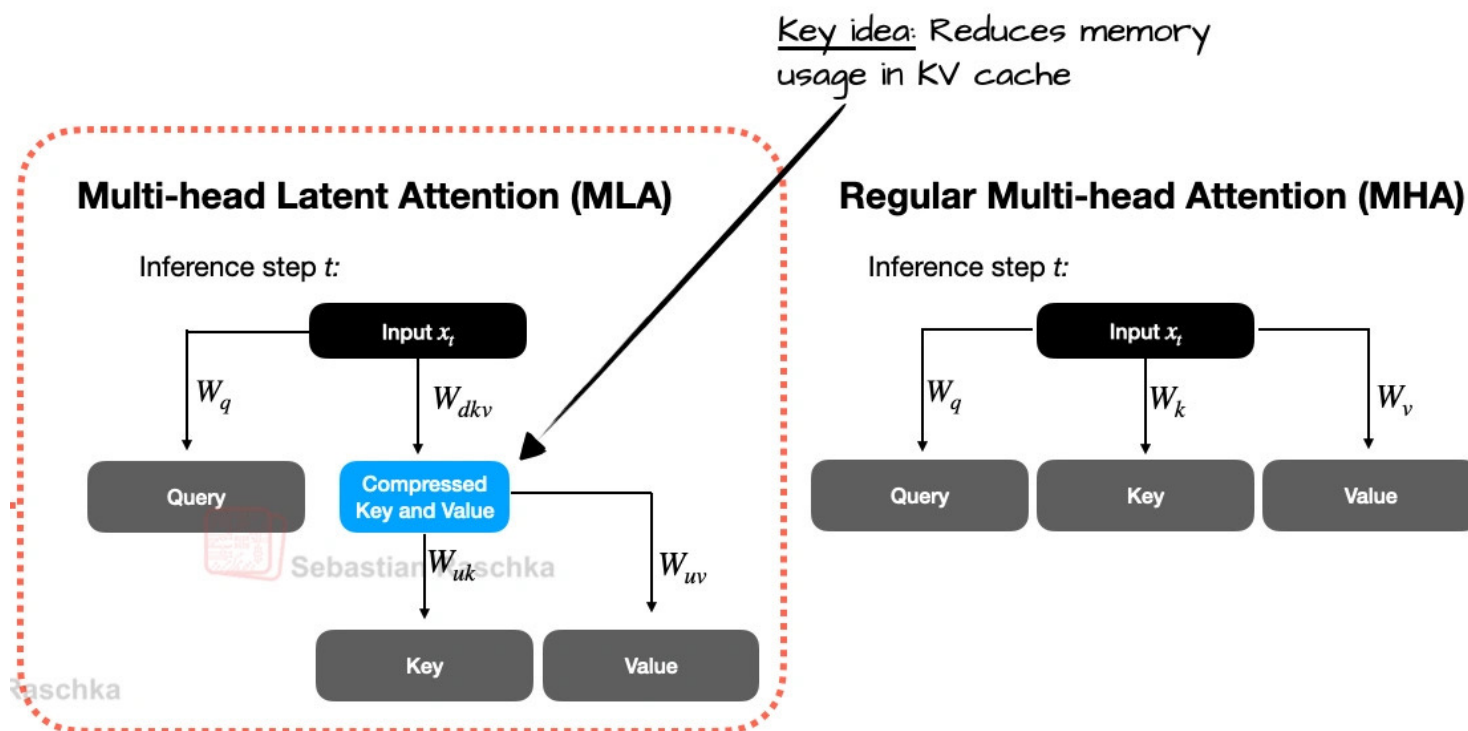
- **Grouped query attention (GQA)**
- Instead of giving every query head its own keys and values, GQA lets several query heads share the same key-value projections.



Example models: Llama3-8B, Qwen3-4B, Gemma3-27B etc.

Improvements to Attention Mechanism

- **Multi-Head Latent Attention (MLA)**
- MLA is similar to GQA, but the difference is that MLA shrinks the cache by *compressing* what gets stored rather than sharing heads.



Example models:
DeepSeek-V3, Kimi-K2,
GLM-5 etc.

Improvements to Attention Mechanism

- **Sliding Window Attention (SWA)**
- SWA reduces the computation cost by limiting the attention window: Instead of attending to the entire prefix, each token only attends to a fixed window of neighbors.

	The	model	attends	to	past	tokens
The	1	0	0	0	0	0
model	1	1	0	0	0	0
attends	1	1	1	0	0	0
to	1	1	1	1	0	0
past	1	1	1	1	1	0
tokens	1	1	1	1	1	1

	The	model	attends	to	past	tokens
The	1	0	0	0	0	0
model	1	1	0	0	0	0
attends	1	1	1	0	0	0
to	0	1	1	1	0	0
past	0	0	1	1	1	0
tokens	0	0	0	1	1	1

Example models:
OLMo3-32B, MiMo-V2-Flash

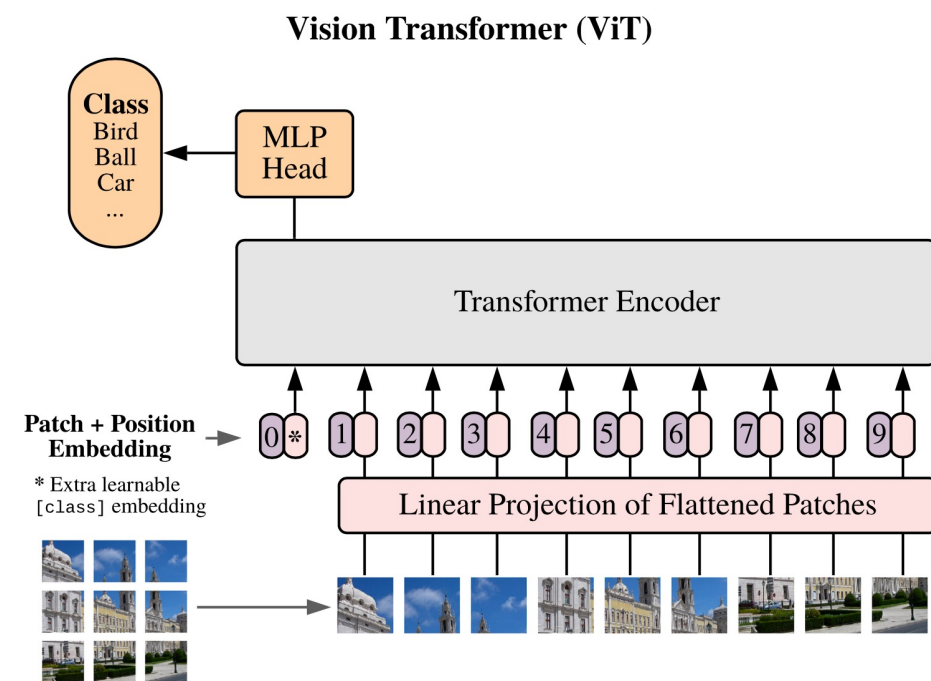
Hardware and system-level optimization

- FlashAttention and v2 (Dao et al., 2022)
 - v1: Reorders attention computation and leverages classical techniques (tiling, recomputation) to speed it up.
 - v2: Better parallelism, work partitioning, and fewer non-matmul FLOPs.
- PagedAttention (Woosuk Kwon et al., 2023)
 - Attention bottleneck: KV-cache -- attention keys and values generated for previous tokens take large GPU memory space.
 - PagedAttention optimizes memory usage by partitioning KV cache into blocks, and access them through a lookup table.

Visual Transformer (ViT)

Dosovitskiy et al., 2020

- Vision Transformer (ViT) is a transformer designed for vision tasks



Vision Transformers

Transformers / Davide Coccozzini / 2021

https://en.wikipedia.org/wiki/Vision_transformer#/media/File:Vision_Transformer.gif

Reference

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Shen, S., Yao, Z., Gholami, A., Mahoney, M., & Keutzer, K. (2020, November). Powernorm: Rethinking batch normalization in transformers. In *International conference on machine learning* (pp. 8741-8751). PMLR.